ISO/IEC JTC 1/SC 22/WG14
WG 21, SG 22

N2890 v4
P2304R3

2021-12-25

Improve type generic programming
proposal for C23

Jens Gustedt
INRIA and ICube, Université de Strasbourg, France

C already has a variaty of interfaces for type-generic programming, but lacks a systematic approach that provides type safety, strong ecapsulation and general usability. This paper is a summary paper for a series that provides improvements through

N2891. type inference for variable definitions (**auto** feature) and function return
N2892. Basic lambdas for C
N2893. Options for lambdas
N2894. type-generic lambdas (with **auto** parameters)

Other papers already build on these at least partially and use lambdas in a wider context.

N2862. Function Pointer Types for Pairing Code and Data
N2895. A simple defer feature for C

The aim is to have a complete set of features that allows to easily specify and reuse type-generic code that can equally be used by applications or by library implementors. All this by remaining faithful to C's efficient approach of static types and automatic (stack) allocation of local variables, by avoiding superfluous indirections and object aliasing, and by forcing no changes to existing ABI.

Changes: v4/R3 update of the new division into "basic" and "options" and apply the changed terminology of "shadow captures" and "identifier captures". Add an outlook to two papers that already build on these features.

v2/R1 and v3/R2 provide updates of the proposed wording. For details of the applied changes see corresponding papers as indicated above.

Contents

I. INTRODUCTION

With the exception of type casts and pointer conversions to and from **void***, C is a programming language with a relatively rigid type system that can provide very useful diagnostics during compilation, if expected and presented types don't match. This rigidity can be, on the other hand, quite constraining when programming general features or algorithms that potentially can apply to a whole set of types, be they pre-defined by the C standard or provided by applications.

This is probably the main reason, why C has no well established general purpose libraries for algorithmic extensions; the interfaces (**bsearch** and **qsort**) that the C library provides are quite rudimentary. By using pointer conversions to **void*** they circumvent exactly the type safety that would be critical for a safe and secure usage of such generic features.

To our knowledge, libraries that provide type-generic features only have a relatively restricted market penetration. In general, they are tedious to implement and to maintain and the interfaces they provide to their users may place quite a burden of consistency checks to these users.

On the other hand, some extensions in C implementations and in related programming languages have emerged that provide type-genericity in a much more comfortable way. At the same time these extensions improve the type-safety of the interfaces and libraries that are coded with them.

An important feature that is proposed here, again, are lambdas. WG14 had talked about them already at several occasions [Garst 2010; Crowl 2010; Hedquist 2016; Garst 2016; Gustedt 2020b], and one reason why their integration in one form or another did not find consensus in 2010 seems to be that, at that time, it had been considered to be too late for C11. An important data point for lambdas is also that within C++ that feature has much evolved since C++11; they have become an important feature in every-day code (not only for C++ but many other programming languages) and their usability has much improved. Thus we think that it is time to reconsider them for integration into C23, which is our first opportunity to add such a new feature to C since C11.

The goal of this paper is to provide an argumentation to integrate some of the existing extensions into the C programming language, such that we can provide interfaces that

— are type and qualifier safe;
— are comfortable to use as if they were just simple functions;
— are comfortable to implement without excessive case analysis.

It provides the introduction to four other papers that introduce different aspects of such a future approach for type-generic programming in C. Most of the features already have been proposed in [Gustedt 2020b] and the intent of these four papers is to make concrete proposals to WG14 for the addition of these features, namely

(1) type inference for variable definitions and function returns,
(2) basic lambdas for C,
(3) options for lambdas,
(4) type-generic lambdas.

Additionally, we also anticipate that the **typeof** feature as proposed by a fifth paper [Meneide 2020], should be integrated into C.

This paper is organized as follows. Below, Section II, we will briefly present these five papers in subsections of their own. In Section III, we will discuss in more detail the 8 features in the C standard that already provide type-genericity. Section IV then discusses the major problems that current type-generic programming in C faces and the missing properties that we would like to achieve with the proposed extensions. Then, Section V introduces the extension that could close the gaps and shows examples of type-generic code using them, and Section VII provides the combinations of all wordings that are proposed by the four papers in the series.

II. A LEVELED SPECIFICATION OF NEW FEATURES

In the following we briefly present the five papers that should be proposed for C23. The first (Section II.1) and the second (Section II.2) handle two forms of type inference. The first uses direct inference from a wide range of features, namely identifiers, type names and expressions, without performing any form of conversion. The second uses inference from evaluated expressions that undergo lvalue conversion, array-to-pointer and function-to-pointer decay. For simplicity, we now derive the second from the first; previous versions of this paper had them independent.

The third paper, Section II.3, introduces a simple version of C++'s lambda feature. In its proposed form it builds on II.2 for (lack of) the specification of return types, but this dependency could be circumvented by adding additional C++ syntax for the specification of return types. On the other hand, without the **auto** feature for variable definitions the lambda feature would loose part of its usefulness, because variables of lambda type could not be declared.

Paper II.4 adds several optional features to II.3.

Paper II.5 builds on II.2 and II.3 to provide quite powerful type-generic lambdas.

II.1. Type inference from identifiers, value expressions and type expressions

Our hope is that the attempts to integrate gcc's **typeof** extension will be successful. We think that a **typeof** operator that has similar syntactic properties as the **sizeof** and **alignof** operators and that maintains all type properties such as qualification and derivation (atomic, array, pointer or function) could be quite useful for type-generic programming and its type safety.

II.2. Type inference for variable definitions (auto feature) and function return

C's declaration syntax currently already allows to omit the type in a variable definition, as long as the variable is initialized and a storage initializer (such as **auto** or **static**) disambiguates the construct from an assignment. In previous versions of C the interpretation of such a definition had been to attribute the type **int**; in current C this is a constraint violation. We will propose to align C with C++, here, and to change this such the type of the variable is inferred the type from the initializer expression. In a second alignment with C++ we will also propose to extend this notion of **auto** type inference to function return types, namely such that such a return type can be deduced from **return** statements or be **void** if there is none.

II.3. Basic lambdas

Since 2011, C++ has a very useful feature called lambdas. These are function-like expressions that can be defined and called at the same point of a program. The simple lambdas that are introduced in this paper are of two kind. We call the first *function literals*, that are lambdas that interact with their context only via the arguments to a call, no automatic variables of the context can be evaluated within the function body. If they are not used in a function call such function literals can be converted to function pointers with the corresponding prototype. The concept is extended with *closures*, namely lambdas that can access all or part of their context, but by evaluating expressions and fixing them in a so-called *value capture* at the same point where the lambda as a whole is evaluated, or, by importing an automatic variable lexically as an *identifier capture*. The return type of any such lambda

is not provided by the interface specification but it is deduced from the arguments to a possible call.

II.4. Options for lambdas

The basic feature introduced so far only allows an explicit access to the environment of a lambda expression. This paper optionally adds *shadow captures* which evaluate a variable at the point of lambda evaluation and shadows all use of the variable within the body of the lambda. Also it adds notation for defaults: `=` for default shadow captures and `&` for default identifier capture.

Additionally this paper also proposes a syntax disambiguation for an ambiguity between lambdas and attributes.

II.5. Type-generic lambdas (with `auto` parameters)

Type-generic lambdas extend the lambda feature such that the parameter types can use the **`auto`** feature and thus be underspecified. This allows lambdas to be a much more general tool and eases the programming of type-generic features. The concrete types of the **`auto`** parameters for a specific instance of such a lambda are deduced either from the arguments if the lambda is used in a function call, or from the target type of a lambda-to-function-pointer conversion.

III. EXISTING TYPE-GENERIC FEATURES IN `C`

Type-generic features are so deeply integrated into `C` that most programmers are probably not even aware of there omnipresence. Below we identify eight different features that do indeed provide type-genericity, ranging from simple features, such as operators that work for multiple types, to complicated programmable features, such as generic primary expressions (**`_Generic`**).

The following discussion is not meant to cover all aspects of existing type-generic features, but to raise awareness for their omnipresence, for their relative complexity, and for their possible defects.

III.1. Operators

The first type-generic feature of `C` are operators. For example the binary operators `==` and `!=` are defined for all wide integer types (**`signed`**, **`long`**, **`long long`** and their unsigned variants), for all floating types (**`float`**, **`double`**, **`long double`** and their complex variants) and for pointer types, see Tab. I for more details.

Table I. Permitted types for binary operators that require equal types

		floating		pointer object		
operator	wide integer	real	complex	complete	**`void`**	function
`==`, `!=`	×	×	×	×	×	×
`-`	×	×	×	×		
`+`, `*`, `/`	×	×	×			
`<`, `<=`, `>=`, `>`	×	×		×		
`%`, `^`, `&`, `|`	×					

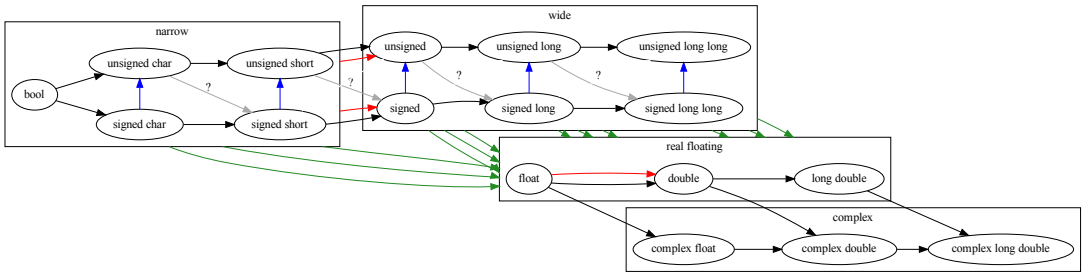

Fig. 1. Upward conversion of arithmetic types. Black arrows conserve values, red arrows may occur for integer promotion or default argument conversion, blue arrows are reduction modulo 2^N, well-definition of grey arrows depends on the platform, green arrows may loose precision

Thus, expressions of the form `a*b+c` are by themselves already type-generic and the programmer does not have to be aware of the particular type of any of the operands. In addition, if the types of the operands do not agree, there is a complicated set of conversions (see below) that enforces equal types for all these operations. Other binary operators (namely shift operators, object pointer addition, array subscripting) can even deal with different operand types, even without conversion.

III.2. Default promotions and conversions

If operands for the operators in Tab. I don't agree, or if they are even types for which these operands are not supported (narrow integer types such as **`bool`**, **`char`** or **`short`**) a complicated set of so-called promotion and conversion rules are set in motion. See Fig. 1 for an overview.

III.2.1. Conversions. Whenever an arithmetic argument to a function or the LHS of an assignment or initialization has not the requested type of the corresponding parameter, there is a whole rule set that provides a conversion from the argument type to the parameter type.

```
1   printf("result is: %g\n", cosf(1));
```

Here, the **`cosf`** function has a **`float`** parameter and so the **`int`** argument `1` is first converted to `1.0f`.

Figure 1 shows the *upward* conversions that are put in place by `C`. These kind of conversions help to avoid to write several versions of the same function and allow to use such a function, to a certain extend, with several argument types.

III.2.2. Promotion and default argument conversion. In the above example, the result of **`cosf`** is **`float`**, too, but **`printf`** as a variadic function cannot handle a **`float`**. So that value is converted to **`double`** before being printed.

Generally, there are certain types of numbers that are not used for arithmetic operators or for certain types of function calls, but are always replaced by a wider type. These mechanisms are called *promotion* (for integer types) or *default argument conversion* (for floating point).

III.2.3. Default arithmetic conversion. To determine the target type of an arithmetic operation, these concepts are taken on a second level. *Default arithmetic conversion* determines a common "super" type for binary arithmetic operators. For example, an operation `-1 + 1U` first performs the minus operation to provide a **`signed int`** of value −1, then (for arithmetic conversion) converts that value to an **`unsigned int`** (with value **`UINT_MAX`**) and performs the addition. The result is an **`unsigned int`** of value 0.

III.3. Macros

C's preprocessor has a powerful macro feature that is designed to replace identifiers (so-called object macros) and pseudo-function calls by other token sequences. Together with default arithmetic promotions it can be used to provide type-generic programming for several categories of tasks:

— type-generic expressions
— type-generic declarations and definitions
— type-generic statements that are not expressions

III.3.1. Macros for type-generic expressions. A typically type-generic macro has an arithmetic expression that is evaluated and that uses default arithmetic conversion to determine a target type. For example the following macro computes a grey value from three color channels:

```
1   #define GREY(R, G, B) (((R) + (G) + (B))/3)
```

It can be used for any type that would be used to represent colors. If used with **`unsigned char`** the result would typically be **`int`**, for **`float`** values the result would also be **`float`**.

Naming conventions, here for structure members `r`, `g`, and `b`, can also help to write type generic macros.

```
1  #define red(P) (P.r)
2  #define green(P) (P.g)
3  #define blue(P) (P.b)
4  #define grey(P) (GREY(P.r, P.g, P.b))
```

III.3.2. Macros for declarations and definitions. Type defitions that then can use the above macros can also be provided by macros.

```
1  #define declareColor(N) typedef struct N N
2
3  declareColor(color8);
4  declareColor(color64);
5  declareColor(colorF);
6  declareColor(colorD);
7
8
9  #define defineColor(N, T) struct N { T r; T g; T b; }
10
11 defineColor(color8, uint8_t);
12 defineColor(color64, uint64_t);
13 defineColor(colorF, float);
14 defineColor(colorD, double);
```

III.3.3. Macros placeable as statements. Macros can also be used to group together several statements for which no value return is expected. Unfortunately, coding properly with this technique usually has to trade in some uglyness and maintenance suffering. The following presents common practice for generic macro programming in C that can be used for any structure type T that has a **mtx_t** member mut and a data member that is assignment compatible with BASE.

```
#define dataCondStore(T, BASE, P, E, D)      \
  do {                                       \
    T* _pr_p = (P);                          \
    BASE _pr_expected = (E);                 \
    BASE _pr_desired = (D);                  \
    bool _pr_c;                              \
    do {                                     \
      mtx_lock(&_pr_p->mtx);                 \
      _pr_c = (_pr_p->data == _pr_expected); \
      if (_pr_c) _pr_p->data = _pr_desired;  \
      mtx_unlock(&_pr_p->mtx);               \
    }  while(!_pr_c);                        \
  } while (false)
```

Coded like that, the macro has several advantages:

— It can syntactically be used in the same places as a **void** function. This is achieved by the crude outer **do** ... **while(false)** loop.
— Macro parameters are evaluated at most once. This is achieved by declaring auxiliary variables to evaluate and hold the values of the macro arguments. Note that the definition of these auxiliary variables needs knowledge about the types T and BASE.
— Some additional auxiliary variables (here _pr_c) can be bound to the scope of the macro.

Additionally, a naming convention for local variables is used as to minimize possible naming conflicts with identifiers that might already be defined in the context where the macro is used. Nevertheless, such a naming convention is not fool proof. In particular, if the use of several such macros is nested, surprising interactions between them may occur.

III.4. Variadic functions

Above we also have seen another C standard tool for type-generic interfaces, variadic functions such as **printf**:

```
  int printf(char const format[static 1], ...);
```

The ... denotes an arbitrary list of arguments that can be passed to the function, and it is mostly up to a convention between the implementor and the user how many and what type of arguments a call to the function may receive. There are notable exceptions, though, because with the ... notation all arguments that are narrow integers or are **float** are converted, see Figure 1.

For such interfaces in the C standard library modern compilers can usually check the arguments against the format string. In contrast to that, user specified functions remain usually unchecked and can present serious safety problems.

III.5. function pointers

Function pointers allow to handle algorithms that can be made dependent of another function. For example, here is a generic function that computes an approximation of the derivative of function `func` in point `x`:

```
1 typedef double math_f(double);
2
3 inline double tangent5(math_f* func, double x, double ε) {
4   double h = ε * x;
5   return (-func(x + 2*h) +8*func(x + h)
6           -8*func(x - h) +func(x - 2*h))/(12*h);
7 }
```

III.6. void pointers

The C library itself has some interfaces that use function pointers for type-genericity, namely **bsearch** and **qsort** receive a function pointer to the following function type

```
1 typedef int compar_t(void const*, void const*);
```

with the understanding that the pointer parameters of such a function represent pointers to the same object type `BASE`, depending on the function, and that the return value is less than, equal to, or greater than `0` if the first argument compares less than, equivalent to, or greater than the second argument.

```
1 int comparDouble(void const* A, void const* B){
2   double const* a = A;
3   double const* b = B;
4   return (*a < *b) ? -1 : ((*a == *b) ? 0 : +1);
5 }
6
7 double tabd[] = { 1, 4, 2, 3, };
8 qsort(tab, sizeof tabd[0], sizeof tab/sizeof tabd[0],
     comparDouble);
```

This uses the fact that data pointers can be converted forth and back to **void** pointers, as long as the target qualification is respected. The advantage is that such a comparison (and thus search or sorting) interface can then be written quickly. The disadvantage is that guaranteeing type safety is solely the job of the user.

III.7. Type-generic C library functions

C gained its first explicit type-generic library interface with the introduction of `<tgmath.h>` in C99. The idea here is that a functionality such as the cosine should be presented to the user as a single interface, a type-generic macro **cos**, instead of the three functions **cos**, **cosf** and **cosl** for **double**, **float** or **long double** arguments, respectively.

At least for such one-argument functions the expectation seems to be clear, that such a functionality should return a value of the same type as the argument. In a sense, such type-generic macros are just the extension of C's operators (which are type-generic) to a set of well specified and understood functions. An important property here is that each of the type-generic macros in `<tgmath.h>` represents a finite set of functions in `<math.h>` or `<complex.h>`. Many implementations implemented these macros by just choosing a function pointer by inspecting the size of the argument, using the fact that their representations of the argument types all had different sizes.

Then, C11 gained a whole new set of type-generic functions in `<stdatomic.h>`. The difficulty here is that there is a possibly unbounded number of atomic types, some of which with equal size but different semantics, and so the type-generic interfaces cannot simply rely on the argument size to map to a finite set of functions. Implementations generally have to rely on language extensions to implement these interfaces.

III.8. `_Generic` primary expressions

C11 introduced a new feature, generic primary expressions, that was primarily meant to implement type generic macros similar to those in `<tgmath.h>`, that is to perform a choice of a limited set of possibilities, guided by the type of an input expression. By that our example for **cos** from above could be implemented as follows:

```
1 #define cos(X)                          \
2   _Generic((X),                         \
3     float: cosf,                        \
4     long double: cosl,                  \
5     default: cos)(X)
```

That is a **_Generic** expression is used to choose a function pointer that is then applied to the argument `X`. Note that here **_Generic** only uses `X` for its type and does not evaluate it, that the result type of the **_Generic** is the type of the chosen expression, and, that the library function **cos** can be used within the macro, because C macros are not recursive. Thus, this technique allows an "overload" of some sort of the function **cos** with the macro **cos**. Another implementation could be as follows:

```
1 #define cos(X)                           \
2   _Generic((X),                          \
3     float: cosf((float)X),               \
4     long double: cosl((long double)X), \
5     default: cos((double)X))
```

By this, **cosf** and **cosl** themselves could even be macros and the compiler would not have to use the corresponding function pointers.

The concept of generic primary expressions goes much further than for switching between different function pointers. For example, the following can do a conversion of a pointer value `P` according to the type of an additional argument `X`.

```
1 #define getOrderCP(X, P)                                  \
2   _Generic((X),                                           \
3     float: (float const*)(P),                             \
4     double: (double const*)(P),                           \
5     long double: (long double const*)(P),                 \
```

```
    unsigned: (unsigned const*)(P),                    \
    unsigned long: (unsigned long const*)(P),          \
    ... /* other ordered arithmetic types */ ...        \
    )
```

Still, the important concepts are the same: X is only used for its type, and the type of the expression itself corresponds to the type of the choosen expression.

IV. MISSING FEATURES

IV.1. Temporary variables of inferred type

One of the most important restrictions for type-generic statements above (III.3.3) was that the macro needed arguments that encoded the types for which the macro was evaluated. This not only inconvenient for the user of these macros but also an important source of errors. If the user chooses the wrong type, implicit conversions can impede on the correctness of the macro. For our example dataCondStore a wrong choice of the type BASE **float** instead of **double** could for example have the effect that the equality test never triggers, and thus that the inner loop never terminates.

In accordance with C's syntax for declarations and in extension of its semantics, C++ has a feature that allows to infer the type of a variable from its initializer expression.

```
auto y = cos(x);
```

This eases the use of type-generic functions because now the return value **and** type can be captured in an auxiliary variable, without necessarily having the type of the argument, here x, at hand. This can become even more interesting if the return type of type-generic functions is just an aggregation of several values for which the type itself is just an artefact:

```
#define div(X, Y)            \
  _Generix((X)+(Y),          \
           int: div,         \
           long: ldiv,       \
           long long: lldiv) \
           ((X), (Y))

auto res = div(38484848448, 448484844);  // int or long?
auto a = b * res.quot + res.rem;
```

Used in the macro from III.3.3, this can easily remove the need for the specification of the types T and BASE:

```
#define dataCondStoreTG(P, E, D)                  \
  do {                                            \
    auto* _pr_p = (P);                            \
    auto _pr_expected = (E);                      \
    auto _pr_desired = (D);                       \
    bool _pr_c;                                   \
    do {                                          \
      mtx_lock(&_pr_p->mtx);                      \
      _pr_c = (_pr_p->data == _pr_expected); \
```

```
10        if (_pr_c) _pr_p->data = _pr_desired;  \
11        mtx_unlock(&_pr_p->mtx);               \
12      }  while(!_pr_c);                        \
13    } while (false)
```

IV.2. Controlled encapsulation

Even as presented now, the macro `dataCondStoreTG` has a serious flaw that is not as apparent as it should be. The assignment of the values of `E` and `D` to `_pr_expected` and `_pr_desired` is not independent. This is, because `D` itself may be an expression that contains a reference to an identifier `_pr_expected`, and thus the intended evaluation of `D` (before even entering the macro) is never performed, but a completely different value (depending on `E`) is used instead.

```
1   dataCondStoreTG(P, 4, 3*_pr_expected);
```

The result of the macro then depends on the order of specification of the variables `_pr_expected` and `_pr_desired`. This kind of interaction is the main reason why we had to chose these ugly names with a `_pr_` prefix in the first place: they reduce the probability of interaction between the code inside the macro and its caller.

C++ has a feature that is called *lambda*. In its simplest form (that we call *function literal*) it provides just the possibility to specify an anonymous function that only interacts with its context via parameters:

```
1  auto const dataCondStoreλDD =
2    [](DD *p, double expected, double desired) {
3      bool c;
4      do {
5        mtx_lock(&p->mtx);
6        c = (p->data == expected);
7        if (c) p->data = desired;
8        mtx_unlock(&p->mtx);
9      }  while(!c);
10   };
11
12   dataCondStoreλDD(pDD, 0.5, 0.7);
```

Here, we may now chose "decent" variable and parameter names, because we know that they will not interact with a calling context.

When we combine lambdas with the **auto** feature for the parameters, this tool becomes even more powerful, because now we have in fact a way to describe a type-generic functionality without having to worry about the particular types of the arguments nor of an uncontrolled interaction with the calling environment.

```
1  #define dataCondStoreλ                          \
2    [](auto *p, auto expected, auto desired) { \
3      bool c;                                   \
4      do {                                      \
5        mtx_lock(&p->mtx);                      \
6        c = (p->data == expected);              \
```

```
      if (c) p->data = desired;                \
      mtx_unlock(&p->mtx);                     \
    }  while(!c);                              \
  }

  dataCondStoreλ(pDD, 0.5, 0.7);
  dataCondStoreλ(pFF, 0.1f, 0);
```

IV.3. Controlled constant propagation

The above form of lambdas for function literals is introduced by an empty pair of brackets `[]` to indicate that the lambda does not access to any automatic variables from the calling context. More general forms of lambdas called *closures* are available in C++ that provide access to the calling context.

The idea is that the body of a closure may use identifiers that are *free*, that is that don't have a definition that is provided by the lambda itself but by the calling context. C++ has a strict policy here, that such free variables must be explicitly named within the brackets, or that the bracket should have a `=` token to allow any such free variables to appear. For example a lambda expression as in the following

```
auto const tangent5λ = [ε](math_f* func, double x) {
  double h = ε * x;
  return (-func(x + 2*h) +8*func(x + h)
          -8*func(x - h) +func(x - 2*h))/(12*h);
};
```

captures the value ε from the environment and freezes it for any use of the `tangent5λ` closure to the value at the point of evaluation of the lambda (and not the call).

An even more extended form of this allows the assignment of any expression to the free variables:

```
#define TANGENT5(F, E) [func = (F), ε = (E)](double x) { \
  double h = ε * x;                                      \
  return (-func(x + 2*h) +8*func(x + h)                  \
          -8*func(x - h) +func(x - 2*h))/(12*h);         \
}

int main(int argc, char* argv[static argc+1]) {
  auto const f0 = argc > 1 ? &sin : &cos; // function pointer
  auto const f1 = TANGENT5(f0, 0x1E-12);  // lambda value
  auto const f2 = TANGENT5(f1, 0x1E-12);  // lambda value
  auto const f3 = TANGENT5(f2, 0x1E-12);  // lambda value

  for (double x = 0.01; x < 4; x += 0.5) {
    printf("%g␣%g␣%g␣%g\n", f0(x), f1(x), f2(x), f3(x));
  }
}
```

Here, three lambdas are evaluated and assigned to **auto** variables `f1`, `f2` and `f3`, respectively. By that technique, the compiler is free to optimize the code in the body of the lambda with

respect to the possible values of `func` and ε, and then to use these optimized versions within the **`for`** loop as indicated.

IV.4. Automatic instantiation of function pointers

Library programmers often need a seamless tool to describe and implement a generic feature, and, from time to time, they need the possibility to instantiate a function pointer for a certain set of function arguments from there. **`_Generic`** provides the complete opposite of that: previously unrelated specialized function pointers are stitched together into one feature.

C++'s lambda model allows to provide such a more practical tool, namely it allows to instantiate function pointers from all function literals.

```
1  auto const sortDouble =
2    // function literal
3    [](size_t len, double const ar[static len]) {
4        // function pointer
5        int (*comp)(void const*, void const*) =
6          // function literal
7          [](void const* A, void const* B){
8            double const* a = A;
9            double const* b = B;
10           // returns -1, 0, or +1, an int
11           return (*a < *b) ? -1 : ((*a == *b) ? 0 : +1);
12         };
13         qsort(ar, sizeof ar[0], len, comp);
14       );
15       // no return statement, void
16   };
17
18 double tabd[] = { 1, 4, 2, 3, };
19 sortDouble(sizeof tab/sizeof tabd[0], tabd);
```

That is, all lambdas without capture can be converted implicitly or explicitly to a function pointer with a prototype that is compatible with the parameter and return types of the lambda. If such an attempt is made and the parameter types are not compatible, an error (constraint violation) occurs and the compilation should abort. In the above example the inner lambda has two parameters of type **`void const*`** and its **`return`** expression has type **`int`**. Thus its lambda type is convertible to the function pointer type as indicated.

Such a conversion to a function pointer can be done implicitly as above, in an intialization, assignment or by passing a lambda as an argument to a function call. It can also come from an explicit conversion, that is a cast operator.

IV.5. Automatic instantiation of specializations

When the parameters of a lambda use the **`auto`** feature, we have a *type-generic* lambda, that is a lambda that can receive different types of parameters. When such a lambda is used, the underspecified parameter types must be completed, such that the compiler can instantiate code that has all types fixed at compile time.

If there are no captures, one possibility to determine the parameter types is to assign such a type-generic lambda to a function pointer:

```
#define TANGENT5TG(auto* func, auto x, auto ε) {  \
  auto h = ε * x;                                 \
  return (-func(x + 2*h) +8*func(x + h)           \
          -8*func(x - h) +func(x - 2*h))/(12*h);  \
}

typedef double math_f(double);
typedef float mathf_f(float);
typedef long double mathl_f(long double);


double (*tangent5)(math_f*, double, double) = TANGENT5TG;
float (*tangent5f)(mathf_f*, float, float) = TANGENT5TG;
long double (*tangent5l)(mathl_f*, long double, long double) =
   TANGENT5TG;
```

Here, again, such a conversion to a function pointer can only be formed if the parameter and return types can be made consistent.

The following shows how an inner lambda can even be made type-generic, such that it synthesizes a function pointer on the fly, whenever the outer lambda is instantiated:

```
#define sortOrder                                          \
    [](size_t len, auto const ar[static len]) {            \
       qsort(ar, sizeof ar[0], len,                        \
          [](void const* A, void const* B){                \
            auto const* a = getOrderCP(ar[0], A);          \
            auto const* B = getOrderCP(ar[0], B);          \
            return (*a < *b) ? -1 : ((*a == *b) ? 0 : +1); \
          }                                                \
       );                                                  \
    }

  void (*sortd)(size_t len, double const ar[static len])
    = sortOrder;
  void (*sortu)(size_t len, unsigned const ar[static len])
    = sortOrder;

  double tabd[] = { 1, 4, 2, 3, };
  // semantically equivalent
  sortOrder(sizeof tab/sizeof tabd[0], tabd);
  sortd(sizeof tab/sizeof tabd[0], tabd);

  unsigned tabu[] = { 1, 4, 2, 3, };
  // semantically equivalent
  sortOrder(sizeof tabu/sizeof tabu[0], tabu);
  sortu(sizeof tabu/sizeof tabu[0], tabu);
```

Here, we use the type-generic macro `getOrderCP` from above which does not evaluate its first argument, `ar[0]` in this case, but only uses it for its type. Remember that the visiblity rules for identifiers from outer scopes are the same as elsewhere, only the *access* to automatic variables is constrained or allowed by the capture clause. Thus, such a use for the type inside the inner lambda is allowed, and provides a lambda that is dependent on the type of `ar[0]`.

IV.6. Direct type inferrence

The possibility of inferring a type via the **auto** feature has the property that it is only possible for an expression that is evaluated in an initializer, and thus it first undergoes lvalue, array-to-pointer or function-to-pointer conversion before the type is determined. In particular, by this mechanism it is not possible to propagate qualifiers (including **_Atomic**) nor to conserve array dimensions.

C++ has the **decltype** operator and many C compilers have a **__typeof__** extension that fills this gap. For the following we assume a **typeof** operator that just captures the type of an expression or typename that is passed as an argument.

```
1  int i;
2  // an array of three int
3  typeof(i) iA[] = { 0, 8, 9, };
4
5  double A[4];
6  typedef typeof(A) typeA;
7  // equivalent definition
8  typedef double typeA[4];
9  // equivalent declaration
10 typeA A;
11 // equivalent declaration
12 typeof(double[4]) A;
13 // mutable array of 4 elements intialized to 0
14 typeof(A) dA = { 0 };
15 // immutable array of 4 elements
16 typeof(A) const cA = { 0, 1, 2, 3, };
17
18 // infer the type of a function
19 typeof(sin) cos;
20 // equivalent declaration
21 double cos(double);
22
23 // infer the type of a function pointer and initialize
24 typeof(sin)*const Δ = cos;
25 // equivalent definition
26 auto*const Δ = cos;
27 // equivalent definition
28 const auto Δ = cos;
29 // equivalent definition
30 double (*const Δ)(double) = cos;
```

In particular, for every declared identifier `id` with external linkage (that is not also thread local) the following redundant declaration can be placed anywhere where a declaration is allowed.

```
1  extern typeof(id) id;
```

A **typeof** operator can be used everywhere where an **typedef** identifier can be used. It can not only applied to type expressions and identifiers as above, but also to any valid expression:

```
1  #define sortOrder                                                  \
2      [](size_t len, auto const ar[static len]) {                    \
3         qsort(ar, sizeof ar[0], len,                                \
4            [](void const* A, void const* B){                        \
5              typeof(ar[0])* a = A;                                  \
6              typeof(ar[0])* b = B;                                  \
7              return (*a < *b) ? -1 : ((*a == *b) ? 0 : +1); \
8            }                                                        \
9         );                                                          \
10     }
11
12   void (*sortd)(size_t len, double const ar[static len])
13     = sortOrder;
14   void (*sortu)(size_t len, unsigned const ar[static len])
15     = sortOrder;
16
17   double tabd[] = { 1, 4, 2, 3, };
18   // semantically equivalent
19   sortOrder(sizeof tab/sizeof tabd[0], tabd);
20   sortd(sizeof tab/sizeof tabd[0], tabd);
21
22   unsigned tabu[] = { 1, 4, 2, 3, };
23   // semantically equivalent
24   sortOrder(sizeof tabu/sizeof tabu[0], tabu);
25   sortu(sizeof tabu/sizeof tabu[0], tabu);
```

By that we are now able to remove the call to `getOrderCP` from the inner lambda expression. The result is a macro `sortOrder` that can be used to sort any array as long as the elements that can be compared with the `<` operator. The only external reference that remains is the `C` library function **`qsort`**. That macro can be used to instantiate a function pointer or it can be used directly in a function call.

V. COMMON EXTENSIONS IN C IMPLEMENTATIONS AND IN OTHER RELATED PROGRAMMING LANGUAGES

In the following we are interested in features that extend current C for type-genericity but with one important restriction:

Features that are proposed imply no ABI changes.

In particular, with the proposed changes we do not intend

— to change the ABI for function pointers,
— to introduce linkage incompatibilities such as mangling,
— to modify the life-time of automatic objects, or
— to introduce other managed storage that is different from automatic storage.

There are a lot of features in the field that would need one or several points from the above, such as C++'s `template` functions or functor classes, Objective C's `__block` storage specifiers, or gcc's callable nested functions. All of these approaches have their merits, and this paper is not written to argue against their integration into C. We simply try first to look into the features that can do without, such that they might be easily adopted by programmers that are used to our concepts and implemented more widely than they already are.

V.1. Type inferrence

Besides the possibility of functional expression, declaring parameters, variables and return values of inferred type is a crucial missing feature for an enhancement of standard C towards type-genericity. This allows to declare local, auxiliary, variables of a type that is deduced from parameters and to return dependent values and types from functional constructs.

We found several existing extensions in C or related languages that allow to infer a type from a given construct. They differ in the way derived type constructions (qualifiers, `_Atomic`, arrays or functions) influence the derived type: C++'s **`auto`** feature and gcc's **`auto_type`**, C++'s **`decltype`**, and gcc's **`typeof`**.

V.1.1. **`auto`** *type inference.* This kind of type inference takes up an idea that already exists in C:

> *A type specification may only have incomplete information, and then is completed by an initializer.*

This is currently possible for array declarations where an incomplete specification of an array bound may be completed by an initializer:

```
double const A[] = { 5, 6, 7, };  // array of 3 elements
double const B[] = { [23] = 0, }; // array of 24 zeroes
```

In fact, the maximum index in the initializer determines the size of the array and thereby completes the array type.

`auto` type inference pushes this further, such that also the base type of an object definition can be inferred from the initializer:

```
auto b = B[0]; // this is double
auto a = A;    // this is double const*
```

Here, the initializer is considered to be an *expression*, thus all rules for evaluation of expressions apply. So, qualifiers and some type derivations are dropped. For example, `b` is **`double`**, the **`const`** is dropped, and `A` on the RHS undergoes array-to-pointer conversion and the inferred type for `a` is **`double const*`** and not **`double const[24]`**.

Since in the places that are interesting here `=` can have the meaning of an assignment operator or of an initializer, constructs as the following could be ambiguous:

```
b = B[0];
a = A;
```

This ambiguity can occur as soon that an attempted declaration has no storage class, therefore C++ extends the use of the keyword **auto** and allows to place it in any declaration that is supposed to be completed by an initializer.

This feature is then extended even further into contexts that don't even have initializers:

— An **auto** declaration of a function return type infers the completed return type from a **return** expression, if there is any, or infers a type of **void**, if there is none.
— An **auto** declaration of a function or lambda parameter infers the completed parameter type from the argument to a function call or from the corresponding parameter in a function-pointer conversion.

V.1.2. The **typeof** *feature.* **typeof** is an extension that has been provided since a long time in multiple compilers. A **typeof** specifier is just a placeholder for a type, similar to a **typedef**. It reproduces the type "as-is" without dropping qualifiers and without decaying functions or arrays. With this feature not only qualifiers and atomics do not get dropped, but they can even be added.

It differs (and complements) the **auto** feature syntactically and semantically. Its general forms are

```
typeof(expression)
typeof(type-name)
```

and these can be substituted at any place where a type name may occur. With the definitions of A and B as above

```
auto b = B[0];                 // this is double
auto a = A;                    // this is double const*
typeof(B[0]) β;                // this is double const
typeof(A) α;                   // this is double const[24]
typeof(double const[24]) γ; // same type
```

So here we see that the expressions B[0] and A do not undergo any conversion and so the qualifier and the array derivation remain in place.

There have been some inconsistencies for the type derivation strategies for this operator in the past, but it seems that recent compilers interpret types that are given as arguments as it is presented above.

V.1.3. The **decltype** *feature.* Since almost a decade C++ has introduced the **decltype** feature which in most aspects that concern the intersection with C is similar to **typeof**.

Conceptually, integration into C would be a bit more difficult than for **auto**. This is because for historic reasons C++ here mixes several concepts in an unfortunate way: for some types of expressions **decltype** has a reference type for others it hasn't. The line of when it does this is not where we would expect it to be for C: most lvalues produce a reference type, but not all of them. In particular, direct identification of variables or functions (by identifier) or of structure or union members leads to direct types, without reference, but surrounding them with an expression that conserves their "lvalueness" adds a reference to the type of the **decltype** specification.

It is quite unusual for C to have the type of an expression depend on surrounding `()`, but unfortunately that ship has sailed in C++. Therefore we prefer that a new operator **typeof** be introduced into both languages that clarifies these aspects and that is designed to have exactly the same properties in both.

V.2. Lambdas

As we have seen above, in C macros can serve for two important type-generic tasks, namely the specification of type-generic expressions and the specification of type-generic functions. But unfortunately they cannot, without extension, be used *in place* to specify functional units that use the whole expressiveness of the language to do their computation.

To illustrate that, consider the simple task of specifying a **max** feature that computes the maximum of two values `x` and `y`. In essence, we would like this to compute the expression

```
1  (x < y ? y : x)
```

regardless of the type of the two values `x` and `y`. As such this is not possible to specify this safely with a macro

```
1  #define BADMAX(X, Y) ((X) < (Y) ? (Y) : (X))
```

because such a macro always evaluates one of the argument twice; once in the comparison and a second time to evaluate the chosen value. As soon as we pass in argument expressions that have side effects (such as `i++` or a function call) these effects could be produced twice and therefore result in surprising behavior for the unaware user of the interface.

Also, when we would mix signed and unsigned arguments, the above formula would not always compute the mathematical maximum value of the two arguments because a negative signed value could be converted to a large positive unsigned value.

Thus, already for a simple type-generic feature such as **max**, we would need the possibility to define local variables that only have the scope of the **max** expression, and for which we may somehow infer the type from the arguments that are passed to **max**.

In a slight abuse of terminology we will borough the term *lambda* from the domain of functional programming to describe a functional feature that is an expression with a *lambda value* of *lambda type*. Several proposals have already been discussed to integrate lambdas into C [Garst 2010; Crowl 2010; Hedquist 2016; Garst 2016].

Basically, a lambda value can be used in two ways

— It can be moved around as values of objects, that is assigned to variables or returned from functions.
— It can replace the function specifier in a function call expression.

In C++'s lambda notation (that we will propose to adopt below) a **max** feature can be implemented as follows

```
1  [](auto x, auto y) {
2    if ((x < 0) != (y < 0)) {
3      x = (x < 0) ? 0 : x;
4      y = (y < 0) ? 0 : y;
5    }
```

```
6     return (x < y ? y : x);
7   }
```

That is, `[]` introduces a lambda expression, `x` and `y` are parameters to the lambda that have an underspecified type (indicated by **`auto`**) and a **`return`** statement in the body of the lambda specifies a return value and, implicitly, a return type. The logic of the **`if`** statement is to capture the case where one of the two parameters is negative and the other is not, and then to replace the negative one with the value zero. Thereby the lambda never converts a negative signed value to a positive unsigned value.

Observe, that this lambda does not access any other identifier than its parameters.

Global identifiers are easy to handle by lambdas as they are handled by any traditional `C` function. For these there are two mechanism in play:

> *visibility.* This regulates which identifiers can be used and which type they have. In particular, visible identifiers can be used in some context (such as **`sizeof`** or **`_Generic`**) without being accessed.
> *linkage.* This regulates how the object or function behind an identifier is accessible. In particular, an object or function with internal linkage is expected to be instantiated in the same translation unit, and one with external linkage may refer to another, yet unknown, translation unit.

We will call a lambda as the above that does not access external identifiers other than global variables or functions a *function literal*. This term is chosen because such an expression can be used like other literals in `C`: all information for the lambda value is available at compilation time. Such function literal can be moved freely within the scope of the identifiers that are used.

V.2.1. Possible syntax. There are several possibilities to specify syntax for lambdas and below we will see three such specifications as they are currently implemented in the field:

— `C++` lambdas,
— Objective `C` blocks,
— `gcc`'s statement expressions.

A fourth syntax had been proposed by us in some discussions in WG14, namely to extend the notion of compound literals to function types. Syntactically this could be quite simple: for a compound literal where the type expression is a function specification, the brace-enclosed initializer would be expected to be a *function body*, just as for an ordinary function. The successful presence of gcc's statement expressions as an extension shows that such an addition could be added to C's syntax tree without much difficulties. But these two approaches also share the same insufficiencies, namely the semantic ambiguity how references to local variables of the enclosing function would resolve.

V.2.2. The design space for captures and closures. For an object `id` with automatic storage duration there is currently not much a distinction between the visibility of `id` and the possibility to access the object through `id`. For the current definition of the language this sufficient, but if lambdas are able to refer to identifiers that correspond to objects with automatic storage duration, things become more complicated. For example, we might want to execute a lambda that accesses a local variable `x` in a context where `x` is hidden by another variable with the same name. So lambdas that access local variables must use a different mechanism to do so.

We call lambdas that access identifiers of the context in which they are evaluated, *closures*, and the identifiers that are such accessed by a closure *captures*. Since lambdas are inherently expressions, within the context of `C` there are several possible interpretations of such a capture. The design space for modeling the capture of local variables with existing `C` features can be described as follows:

(1) The identifier `id` of type τ is evaluated at the point of evaluation of the capture, and the value ν of type τ' that is determined is used in place throughout the whole lifetime of the closure, a new feature representing the captured value, shadows the use of the variable throughout the body of the lambda. We call such a capture a *shadow capture*.
If τ would be an array type it would not be copyable (there is no such thing as an array value in `C`) and thus it would not fit well in the scheme of a shadow capture. Therefore, generally array types (and maybe other, non-copyable, types) are not allowed as shadow captures.
A shadow capture can in principle be made visible with three different models as follows. They all have in common that the original object `id` can never appear where a modifiable lvalue is required, such as the LHS of an assignment or as the operand of an increment.

> *rvalue shadow capture.* A shadow capture `id` can be presented as an "rvalue", that is as if it were defined as the result of an expression evaluation `(0,id)`. The address of a capture in this model cannot be taken. Although this might seem the most natural view for the evaluation of lambda expression in `C`, we are not aware of an implementation that that uses this model.
>
> *immutable shadow capture.* A shadow capture `id` is a lambda-local object of type τ'' that is initialized with ν, where τ'' is τ' with an additional **`const`**-qualification. The address of such a capture can be taken and, for example, be passed as argument to a function call. But nevertheless the underlying object cannot be modified.
>
> *mutable shadow capture.* A shadow capture `id` is a lambda-local object of type τ' that is initialized with ν. Such a capture behaves very similar to a function parameter that receives the same value as argument on each function call. Such an object is mutable during the execution of the closure, but all changes are lost as soon as control is returned to the calling context.

Note that because τ' is a type after an evaluation, in all these models qualification or atomicity of τ is dropped.

(2) Throughout the life-time of the closure, `id` refers to the same object that is visible by this name at the point of evaluation of the closure. We call such a capture an *identifier capture*. Since identifier captures refer to objects, the corresponding closure cannot have a life-time that exceeds any of its identifier captures. Since `id` is not evaluated at the same time as the lambda expression is formed, it has the same type τ inside the body of the lambda. No qualifiers are dropped, type derivations such as atomic or array are maintained.

V.2.3. C++ lambdas. `C++` lambdas are the most general existing extension and they also fit well into the constraints that we have set ourselves above, namely to be compatible with existing storage classes. Their syntactic form if we don't consider the possibility of adding attributes is

[*capture-list* **]** $\{$**(** *parameter-list* **)**$\}_{opt}$ **mutable**$_{opt}$ $\{$**->** *return-type*$\}_{opt}$ *function-body*

Identifiers with automatic storage duration are captured exclusively if they are listed in the *capture-list* or if a default capture is given. C++ has two additional forms of captures that add an evaluation in the form of "= *expression*" to the definition of the capture. This leads to the notions of *value capture* and *object alias* that can be used to freeze a particular value or reference for any use of the so-determined lambda. This is a list of captures, each of one the following forms

explicit	
`id`	immutable shadow capture
`id` = *expression*	value capture with type and value of *expression*
`&id`	identifier capture
`&id` = *lvalue-expression*	object alias refering to the same lvalue
default	
	forbidden
`=`	immutable shadow capture
`&`	identifier capture

If the optional keyword **mutable** is present, all captures that would otherwise be immutable shadow captures are mutable shadow captures, instead. If `->` *return-type* is present it describes the return type of the lambda; if not, the return type is deduced from a **return** statement if there is any, or it is **void** otherwise. The *object alias* feature introduces a `C++` reference variable. For `C`, these constructs would need some avoidable extension to the syntax and object semantic, so we will not use these parts of the syntax in the proposed addition to `C`.

The *parameter-list* can be a usual parameter list with the notable extension that the type of a parameter can be underspecified by using the **auto** feature, see below. A lambda that has at least one underspecified parameter is a *type-generic* lambda.

Lambda values can be used just as function designators as the left operand of a function call, and all rules for arguments to such a call and the rules to convert them transfers naturally to a lambda call.

When used outside the LHS of a function call expression, lambdas are just values of some object type that is not further specified. Such a lambda type has no declaration syntax, and so the only way to store a lambda value into an object is to use the **auto** feature:

```
1  auto const λ = [](double x){ return x+1; };
```

By these precautions, for any `C++` lambda the original expression that defined the value is always known. So the compiler will always master any aspects of the lambda, in particular which variables of the context are used as captures. If a lambda value leaves the scope of definition of any of its identifier captures the compiler can print a diagnosis.

Function literals are special with respect to these aspects, since they do not have any captures. This is why these special lambdas allow for a third operation, they can be converted to a function pointer:

```
1  double (*λp)(double) = λ;
2  double (*κp)(double) = [](double x){ return x+1; };
```

V.2.4. Objective C's blocks. Objective C [ObjectiveC 2014] has a highly evolved lambda feature that they call *block*, see also [Garst 2009; Garst 2016]. Their syntax is

$$\texttt{\^{}}\ \textit{return-type}_{opt}\ \{\texttt{(}\ \textit{parameter-list}\ \texttt{)}\}_{opt}\ \textit{function-body}$$

Besides the obvious syntactic difference, blocks lack an important feature of C++ lambdas, namely the possibility to specify the policy for captures. If used without other specific extensions, an Objective C block has the same semantic as a C++ closure with default shadow captures, where any automatic variable in the surrounding context can be used as immutable shadow capture. Such a block can be equivalently defined with a C++ lambda as

$$\texttt{[=]}\ \{\texttt{(}\ \textit{parameter-list}\ \texttt{)}\}_{opt}\ \{\texttt{->}\ \textit{return-type}\}_{opt}\ \textit{function-body}$$

and in particular the variants that omit the return type have a syntax that only differs on the token sequence that introduces the feature:

$$\begin{aligned}\texttt{\^{}}\ &\{\texttt{(}\ \textit{parameter-list}\ \texttt{)}\}_{opt}\ \textit{function-body}\\ \texttt{[=]}\ &\{\texttt{(}\ \textit{parameter-list}\ \texttt{)}\}_{opt}\ \textit{function-body}\end{aligned}$$

An important difference arises though, when it comes to identifier captures, where Objective C takes a completely different approach than C++. Here, the property if a capture is a shadow or an identifier capture is attributed to the underlying variable itself, not to the closure that uses it.

A new storage class for managed storage is introduced, unfortunately also called `__block`; `__block` variables are always identifier captures. Such variables have a lifetime that is prolonged even after their defining scope is left, as long as there is any living closure that refers to it. By this, blocks elegantly resolve the lifetime issues of closures with identifier captures in C++: by definition a block will never access a variable after its end-of-life. This elegance comes at the cost of introducing a new storage class with a substantial implementation cost, a certain runtime overhead, and a lack of expressiveness for the choice of the access model for each individual capture.

Because of this extension of the lifetime of identifier captures, for Objective C it is also much easier to describe functors as variables of block type. The declaration syntax for these is similar to function pointers, but using a ^ token instead of *.

V.2.5. Statement expressions. Statement expressions are an intuitive extension first introduced by the gcc compiler framework. Their basic idea is to surround a compound statement with parenthesis and thereby to transform such a compound statement into an expression. The value of such an expression is the value of the last statement if that is an expression statement, or **void** if it is any other form of statement. With *statements* any list of C statements (including a terminating ; if necessary), the syntax

$$\texttt{(\{}\ \textit{statements}\ \textit{expression}\texttt{; \})}$$

is equivalent to the following function call with a C++ closure with default identifier captures as the left operand

$$\texttt{[\&] (void) \{}\ \textit{statements}\ \texttt{return}\ \textit{expression}\texttt{; \}\ ()}$$

V.2.6. Nested functions. Gcc and related compiler platforms also implement the feature of a *nested function*, that is a function that is declared inside the function body of another function. Obviously, because they are not expressions, nested functions are not lambdas, but we will see below how they can be effectively used to implement lambdas. On the other hand, since they cannot be forward-declared, lambda expressions don't allow for recursion, so nested functions clearly are more expressive.

Nested functions can also capture local variables of the surrounding scope. Because they are not expressions but definitions, the most natural semantic is that of identifier captures for the use of such variables, and this is the semantic that `gcc` applies.

Much as global standard `C` functions, nested functions decay into function pointers if they are used other than for the LHS of a function call. This is even for functions that need access to captures, and thus the ABI must be extended to make this possible. The `gcc` implementation does that by creating a so-called *trampoline* as an automatic object, namely as a small function that collects the local information that is necessary and then calls a conventional function to execute the specified function body. Doing so needs execute rights for the automatic storage in question, which is widely criticized because of its possible security impact. On the other hand, this approach is uncritical when it is used without captures, because then the result of the conversion is a simple, conventional, function pointer.

Provided we have an **`auto`** feature as presented in Section V.1.1 and a **`typeof`** feature as in Section V.1.2, the semantics of a wide variety of `C++` lambdas can be implemented with nested functions. For example, with the `shnell` source-to-source rewriting tool [Gustedt 2020a], we have implemented such a transformation as follows. For a closure of the form

`[id0 =` *expr0*`,` **`...,`** **`idk =`** *exprk*`]` **`(`** *parameter-list* **`)`** *function-body0*

a definition of a state type **`_Uniq_Struct`**, state variable **`_Uniq_Capt`** and a definition of a local function **`_Uniq_Func`** are placed inside the closest compound statement that contains the lambda expression:

```
struct _Unique_Struct {
  typeof(expr0) id0;
  ...
  typeof(exprk) idk;
} _Uniq_Capt;
auto _Uniq_Func( parameter-list ) function-body1
```

Here, *function-body1* is the same as *function-body0*, only that the contents is prefixed with definitions of the captures:

```
auto const id0 = _Uniq_Capt.id0;
...
auto const idk = _Uniq_Capt.idk;
```

The lambda expression itself then has to be replaced by an expression that evaluates all the expressions to be captured, followed by the name of the function:

`((_Uniq_Capt = (struct _Uniq_Struct){` *expr0, ..., exprk* **`}), _Uniq_Func)`**

Similarly to the above, a shadow capture for an identifier **`idI`** can just use **`idI`** itself instead of an expression *exprI*. In the definition of the structure this reads

```
  typeof((void)0, idI) idI;
```

and in the replacement of the call this is

((_Uniq_Capt = (struct _Uniq_Struct){ *..., idI, ...* **}), _Uniq_Func)**

Additionally, a C++ closure that has either a default **&** token or individual identifier captures **&idI** can be implemented by just removing these elements from the capture list. Then, the same restrictions for the lifetime of identifier captures and lambda values applies to the rewritten code, and it is up to the programmer to verify this property.

Although this approach covers a wide range of C++ lambdas, such a rewriting strategy has some limits:

— The lambda expression cannot be used in all places that are valid for expression. This are for example an initializer for a variable that is not the first declared variable in a declaration or a controlling expression of a **for** loop.
— The default token **=** in the capture list is not implementable by such simple rewriting,
— The function body is not checked for an access of automatic variables that are not listed in the capture clause.

VI. OUTLOOK INTO PROPOSALS THAT BUILD ON THIS SERIES

VI.1. Function Pointer Types for Pairing Code and Data

Much care has been taken for this series of papers to only include features that do not need ABI changes, in particular by amending the function call operator to accept lambdas without a transition via a function pointer.

Paper N2862 makes a general proposal that allows to interface calling conventions from C that may come from other programming languages and that combine a function and a specific state, a so-called wide function. One possible application of that new ABI would be lambdas; they could be made to convert to a new pointer type, a pointer to a **wide** function. For example

```
1 int (*compare)(double const*) wide
2   = [b = complicated(43)](double const*a) {
3     return (*a < b) ? -1 : ((*a == b) ? 0 : +1);
4   };
```

or if the lambda expression is type-generic

```
1 int (*compare)(double const*) wide
2   = [b = complicated(43)](auto const*a) {
3     return (*a < b) ? -1 : ((*a == b) ? 0 : +1);
4   };
```

If additionally also default captures would be accepted to C23, gcc's nested functions would have a full equivalent. For example a recursive local function wsum that uses automatic variables a and b could be written as follows.

```
1   auto (*wsum)(size_t n, T A[n]) wide
```

```
    = [&](size_t n, T A[n]) {
      switch(n) {
        case 0:
           return +((T)0);
        case 1:
           return a*A[0];
        default:
           return a*wsum(n/2, A) + b*wsum(n - n/2, &A[n/2]);
      }
    };
```

VI.2. A simple defer feature for C

Paper N2589 has proposed a new defer feature to C, that allows to specify code that is executed whenever a specific block or function is left, even under exceptional circumstances. This feature found support by WG14, but the overall mechanism was perceived as complex and left several design questions open that were delegated to a discussion a TS.

Paper N2895 takes the easy parts of this and models defer as *defer declarations* by means of lambdas. By using lambdas as initializers to unnamed *defer callbacks* it thus avoids most of the design difficulties of the original paper.

References

Lawrence Crowl. 2010. *Comparing Lambda in C Proposal N1451 and C++ FCD N3092*. Technical Report N1483. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1483.htm.

Blaine Garst. 2009. *Apple's extensions to C*. Technical Report N1370. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1370.pdf.

Blaine Garst. 2010. *Blocks proposal*. Technical Report N1451. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1451.pdf.

Blaine Garst. 2016. *A Closure for C*. Technical Report N2030. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2030.pdf.

Jens Gustedt. 2020a. *C source-to-source compiler enhancement from within*. Research Report RR-9375. INRIA. https://hal.inria.fr/hal-02998412

Jens Gustedt. 2020b. *A Common* `C/C++` *Core Specification, rev. 2*. Technical Report N2522. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2522.pdf.

Barry Hedquist. 2016. *WG14 Minutes, Kona, HI, USA, 26-29 October, 2015*. Technical Report N2093. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2093.pdf.

JeanHeyd Meneide. 2020. *Not-So-Magic* - `typeof(...)` *in C*. Technical Report N2593. ISO. available at http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2593.htm.

ObjectiveC 2014. Programming with Objective-C. Apple Inc., https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html.

VII. PROPOSED WORDING

This is the proposed text for the whole series of papers, only missing the optional modifications. It is given as diff against C17. A factored diff for the specific concerns is provided with each individual paper.

— Additions to the text are marked as shown.
— Deletions of text are marked as ~~shown~~.

6. Language

6.1 Notation

1 In the syntax notation used in this clause, syntactic categories (nonterminals) are indicated by *italic type*, and literal words and character set members (terminals) by **bold type**. A colon (:) following a nonterminal introduces its definition. Alternative definitions are listed on separate lines, except when prefaced by the words "one of". An optional symbol is indicated by the subscript "opt", so that

{ *expression*$_{\text{opt}}$ }

indicates an optional expression enclosed in braces.

2 When syntactic categories are referred to in the main text, they are not italicized and words are separated by spaces instead of hyphens.

3 A summary of the language syntax is given in Annex A.

6.2 Concepts

6.2.1 Scopes of identifiers

1 An identifier can denote an object; a function; a tag or a member of a structure, union, or enumeration; a typedef name; a label name; a macro name; or a macro parameter. The same identifier can denote different entities at different points in the program. A member of an enumeration is called an *enumeration constant*. Macro names and macro parameters are not considered further here, because prior to the semantic phase of program translation any occurrences of macro names in the source file are replaced by the preprocessing token sequences that constitute their macro definitions.

2 For each different entity that an identifier designates, the identifier is *visible* (i.e., can be used) only within a region of program text called its *scope*. Different entities designated by the same identifier either have different scopes, or are in different name spaces. There are four kinds of scopes: function, file, block, and function prototype. (A *function prototype* is a declaration of a function that declares the types of its parameters.)

3 A label name is the only kind of identifier that has *function scope*. It can be used (in a **goto** statement) ~~anywhere~~ in the function body in which it appears, and is declared implicitly by its syntactic appearance (followed by a : and a statement). Each function body has a function scope that is separate from the function scope of any other function body. In particular, a label is visible in exactly one function scope (the innermost function body in which it appears) and distinct function bodies may use the same identifier to designate different labels.[29)]

4 Every other identifier has scope determined by the placement of its declaration (in a declarator or type specifier). If the declarator or type specifier that declares the identifier appears outside of any block or list of parameters, the identifier has *file scope*, which terminates at the end of the translation unit. If the declarator or type specifier that declares the identifier appears inside a block or within the list of parameter declarations in a function definition, the identifier has *block scope*, which terminates at the end of the associated block. If the declarator or type specifier that declares the identifier appears within the list of parameter declarations in a function prototype (not part of a function definition), the identifier has *function prototype scope*, which terminates at the end of the function declarator.[30)] If an identifier designates two different entities in the same name space, the scopes might overlap. If so, the scope of one entity (the *inner scope*) will end strictly before the scope of the other entity (the *outer scope*). Within the inner scope, the identifier designates the entity declared in the inner scope; the entity declared in the outer scope is *hidden* (and not visible) within the inner scope.

[29)] As a consequence, it is not possible to specify a **goto** statement that jumps into or out of a lambda or into another function.

[30)] Identifiers that are defined in the parameter list of a lambda expression do not have prototype scope, but a scope that comprises the whole body of the lambda.

5 Unless explicitly stated otherwise, where this document uses the term "identifier" to refer to some entity (as opposed to the syntactic construct), it refers to the entity in the relevant name space whose declaration is visible at the point the identifier occurs.

6 Two identifiers have the *same scope* if and only if their scopes terminate at the same point.

7 Structure, union, and enumeration tags have scope that begins just after the appearance of the tag in a type specifier that declares the tag. Each enumeration constant has scope that begins just after the appearance of its defining enumerator in an enumerator list. An identifier that has an underspecified definition and that designates an object, has a scope that starts at the end of its initializer and from that point extends to the whole translation unit (for file scope identifiers) or to the whole block (for block scope identifiers); if the same identifier declares another entity with a scope that encloses the current block, that declaration is hidden as soon as the inner declarator is met.[31] An identifier that designates a function with an underspecified definition has a scope that starts after the lexically first **`return`** statement in its function body or at the end of the function body if there is no such **`return`**, and from that point extends to the whole translation unit. Any other identifier has scope that begins just after the completion of its declarator.

8 As a special case, a type name (which is not a declaration of an identifier) is considered to have a scope that begins just after the place within the type name where the omitted identifier would appear were it not omitted.

9 **NOTE** Properties of the feature to which an identifier refers are not necessarily uniformly available within its whole scope of visibility. Examples are identifiers of objects or functions with an incomplete type that is only completed in a subscope of its visibility, labels that are only valid targets of **`goto`** statements when the jump does not cross the scope of a VLA, identifiers of objects to which the access is restricted in specific contexts such as signal handlers or lambda expressions, or library features such as **`setjmp`** where the use is restricted to a specific subset of the grammar.

Forward references: declarations (6.7), function calls (6.5.2.2), lambda expressions (6.5.2.6), function definitions (6.9.1), identifiers (6.4.2), macro replacement (6.10.3), name spaces of identifiers (6.2.3), source file inclusion (6.10.2), statements and blocks (6.8).

6.2.2 Linkages of identifiers

1 An identifier declared in different scopes or in the same scope more than once can be made to refer to the same object or function by a process called *linkage*.[32] There are three kinds of linkage: external, internal, and none.

2 In the set of translation units and libraries that constitutes an entire program, each declaration of a particular identifier with *external linkage* denotes the same object or function. Within one translation unit, each declaration of an identifier with *internal linkage* denotes the same object or function. Each declaration of an identifier with *no linkage* denotes a unique entity.

3 If the declaration of a file scope identifier for an object or a function contains the storage-class specifier **`static`**, the identifier has internal linkage.[33]

4 For an identifier declared with the storage-class specifier **`extern`** in a scope in which a prior declaration of that identifier is visible,[34] if the prior declaration specifies internal or external linkage, the linkage of the identifier at the later declaration is the same as the linkage specified at the prior declaration. If no prior declaration is visible, or if the prior declaration specifies no linkage, then the identifier has external linkage.

5 If the declaration of an identifier for a function has no storage-class specifier, its linkage is determined exactly as if it were declared with the storage-class specifier **`extern`**. If the declaration of an identifier for an object has file scope and no storage-class specifier or only the specifier **`auto`** , its linkage is external.

6 The following identifiers have no linkage: an identifier declared to be anything other than an object or a function; an identifier declared to be a function parameter; a block scope identifier for an object declared without the storage-class specifier **`extern`**.

[31] That means, that the outer declaration is not visible for the initializer.

[32] There is no linkage between different identifiers.

[33] A function declaration can contain the storage-class specifier **`static`** only if it is at file scope; see 6.7.1.

[34] As specified in 6.2.1, the later declaration might hide the prior declaration.

7 If, within a translation unit, the same identifier appears with both internal and external linkage, the behavior is undefined.

8 **NOTE** Internal and external linkage is used to access objects or functions that have a lifetime of the whole program execution. It is therefore usually determined before the execution of a program starts. For variables with a lifetime that is not the whole program execution and that are accessed from lambda expressions an additional mechanism called identifier capture is available that dynamically provides the access to the current instance of such a variable within the active function call that defines it.

Forward references: storage durations of objects (6.2.4), declarations (6.7), expressions (6.5), external definitions (6.9), statements (6.8).

6.2.3 Name spaces of identifiers

1 If more than one declaration of a particular identifier is visible at any point in a translation unit, the syntactic context disambiguates uses that refer to different entities. Thus, there are separate *name spaces* for various categories of identifiers, as follows:

— *label names* (disambiguated by the syntax of the label declaration and use);

— the *tags* of structures, unions, and enumerations (disambiguated by following any[35] of the keywords **`struct`**, **`union`**, or **`enum`**);

— the *members* of structures or unions; each structure or union has a separate name space for its members (disambiguated by the type of the expression used to access the member via the `.` or `->` operator);

— all other identifiers, called *ordinary identifiers* (declared in ordinary declarators or as enumeration constants).

Forward references: enumeration specifiers (6.7.2.2), labeled statements (6.8.1), structure and union specifiers (6.7.2.1), structure and union members (6.5.2.3), tags (6.7.2.3), the **`goto`** statement (6.8.6.1).

6.2.4 Storage durations of objects

1 An object has a *storage duration* that determines its lifetime. There are four storage durations: static, thread, automatic, and allocated. Allocated storage is described in 7.22.3.

2 The *lifetime* of an object is the portion of program execution during which storage is guaranteed to be reserved for it. An object exists, has a constant address,[36] and retains its last-stored value throughout its lifetime.[37] If an object is referred to outside of its lifetime, the behavior is undefined. The value of a pointer becomes indeterminate when the object it points to (or just past) reaches the end of its lifetime.

3 An object whose identifier is declared without the storage-class specifier **`_Thread_local`**, and either with external or internal linkage or with the storage-class specifier **`static`**, has *static storage duration*. Its lifetime is the entire execution of the program and its stored value is initialized only once, prior to program startup.

4 An object whose identifier is declared with the storage-class specifier **`_Thread_local`** has *thread storage duration*. Its lifetime is the entire execution of the thread for which it is created, and its stored value is initialized when the thread is started. There is a distinct object per thread, and use of the declared name in an expression refers to the object associated with the thread evaluating the expression. The result of attempting to ~~indirectly~~ access an object with thread storage duration from a thread other than the one with which the object is associated is implementation-defined.

5 An object whose identifier is declared with no linkage and without the storage-class specifier **`static`** has *automatic storage duration*, as do some compound literals. The result of attempting to ~~indirectly~~ access an object with automatic storage duration from a thread other than the one with which the object is associated is implementation-defined.

---

[35] There is only one name space for tags even though three are possible.

[36] The term "constant address" means that two pointers to the object constructed at possibly different times will compare equal. The address can be different during two different executions of the same program.

[37] In the case of a volatile object, the last store need not be explicit in the program.

20 Any number of *derived types* can be constructed from the object and function types, as follows:

— An *array type* describes a contiguously allocated nonempty set of objects with a particular member object type, called the *element type*. The element type shall be complete whenever the array type is specified. Array types are characterized by their element type and by the number of elements in the array. An array type is said to be derived from its element type, and if its element type is *T*, the array type is sometimes called "array of *T*". The construction of an array type from an element type is called "array type derivation".

— A *structure type* describes a sequentially allocated nonempty set of member objects (and, in certain circumstances, an incomplete array), each of which has an optionally specified name and possibly distinct type.

— A *union type* describes an overlapping nonempty set of member objects, each of which has an optionally specified name and possibly distinct type.

— A *function type* describes a function with specified return type. A function type is characterized by its return type and the number and types of its parameters. A function type is said to be derived from its return type, and if its return type is *T*, the function type is sometimes called "function returning *T*". The construction of a function type from a return type is called "function type derivation".

— A *lambda type* is an object type that describes the value of a lambda expression. A complete lambda type is characterized but not determined by a return type that is inferred from the function body of the lambda expression, and by the number, order, and type of parameters that are expected for function calls, and by the lexical position of the lambda expressions in the program; the function type that has the same return type and list of parameter types as the lambda is called the *prototype* of the lambda. A lambda type has no syntax derivation[50] and the lexical position of the originating lambda expression determines its scope of visibility. Objects of such a type shall only be defined as a capture (of another lambda expression) or by an underspecified declaration for which the lambda type is inferred.[51] An object of lambda type shall only be modified by simple assignment (6.5.16.1). A lambda expression that has underspecified parameters has an incomplete lambda type that can be completed by function call arguments, or, if it has no captures, in a conversion to a function pointer.

— A *pointer type* may be derived from a function type or an object type, called the *referenced type*. A pointer type describes an object whose value provides a reference to an entity of the referenced type. A pointer type derived from the referenced type *T* is sometimes called "pointer to *T*". The construction of a pointer type from a referenced type is called "pointer type derivation". A pointer type is a complete object type.

— An *atomic type* describes the type designated by the construct **_Atomic**(*type-name*). (Atomic types are a conditional feature that implementations need not support; see 6.10.8.3.)

These methods of constructing derived types can be applied recursively.

21 Arithmetic types and pointer types are collectively called *scalar types*. Array and structure types are collectively called *aggregate types*.[52]

22 An array type of unknown size is an incomplete type. It is completed, for an identifier of that type, by specifying the size in a later declaration (with internal or external linkage). A structure or union type of unknown content (as described in 6.7.2.3) is an incomplete type. It is completed, for all declarations of that type, by declaring the same structure or union tag with its defining content later in the same scope.

[50] Not even a **typeof** type specifier with lambda type can be formed. So there is no syntax to make a lambda type a choice in a generic selection other than **default**

[51] Another possibility to create an object that has an effective lambda type is to copy a lambda value into allocated storage via simple assignment.

[52] Note that aggregate type does not include union type because an object with union type can only contain one member at a time.

complex result value is a positive zero or an unsigned zero.

2 When a value of complex type is converted to a real type other than **_Bool**,[68] the imaginary part of the complex value is discarded and the value of the real part is converted according to the conversion rules for the corresponding real type.

6.3.1.8 Usual arithmetic conversions

1 Many operators that expect operands of arithmetic type cause conversions and yield result types in a similar way. The purpose is to determine a *common real type* for the operands and result. For the specified operands, each operand is converted, without change of type domain, to a type whose corresponding real type is the common real type. Unless explicitly stated otherwise, the common real type is also the corresponding real type of the result, whose type domain is the type domain of the operands if they are the same, and complex otherwise. This pattern is called the *usual arithmetic conversions*:

> First, if the corresponding real type of either operand is **long double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **long double**.
>
> Otherwise, if the corresponding real type of either operand is **double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **double**.
>
> Otherwise, if the corresponding real type of either operand is **float**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **float**.[69]
>
> Otherwise, the integer promotions are performed on both operands. Then the following rules are applied to the promoted operands:
>
> > If both operands have the same type, then no further conversion is needed.
> >
> > Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank is converted to the type of the operand with greater rank.
> >
> > Otherwise, if the operand that has unsigned integer type has rank greater or equal to the rank of the type of the other operand, then the operand with signed integer type is converted to the type of the operand with unsigned integer type.
> >
> > Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, then the operand with unsigned integer type is converted to the type of the operand with signed integer type.
> >
> > Otherwise, both operands are converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

2 The values of floating operands and of the results of floating expressions may be represented in greater range and precision than that required by the type; the types are not changed thereby.[70]

6.3.2 Other operands

6.3.2.1 Lvalues, arrays, function designators and lambdas

1 An *lvalue* is an expression (with an object type other than **void**) that potentially designates an object;[71] if an lvalue does not designate an object when it is evaluated, the behavior is undefined. When an object is said to have a particular type, the type is specified by the lvalue used to designate

[68] See 6.3.1.2.

[69] For example, addition of a **double _Complex** and a **float** entails just the conversion of the **float** operand to **double** (and yields a **double _Complex** result).

[70] The cast and assignment operators are still required to remove extra range and precision.

[71] The name "lvalue" comes originally from the assignment expression `E1 = E2`, in which the left operand `E1` is required to be a (modifiable) lvalue. It is perhaps better considered as representing an object "locator value". What is sometimes called "rvalue" is in this document described as the "value of an expression".

An obvious example of an lvalue is an identifier of an object. As a further example, if `E` is a unary expression that is a pointer to an object, `*E` is an lvalue that designates the object to which `E` points.

the object. A *modifiable lvalue* is an lvalue that does not have array type, does not have an incomplete type, does not have a const-qualified type, and if it is a structure or union, does not have any member (including, recursively, any member or element of all contained aggregates or unions) with a const-qualified type.

2 Except when it is the operand of the **typeof** specifier, the **sizeof** operator, the unary & operator, the `++` operator, the `--` operator, or the left operand of the `.` operator or an assignment operator, an lvalue that does not have array type is converted to the value stored in the designated object (and is no longer an lvalue); this is called *lvalue conversion*. If the lvalue has qualified type, the value has the unqualified version of the type of the lvalue; additionally, if the lvalue has atomic type, the value has the non-atomic version of the type of the lvalue; otherwise, the value has the type of the lvalue. If the lvalue has an incomplete type and does not have array type, the behavior is undefined. If the lvalue designates an object of automatic storage duration that could have been declared with the **register** storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

3 Except when it is the operand of the **typeof** specifier, the unary **sizeof** operator, or the unary & operator, or is a string literal used to initialize an array, an expression that has type "array of *type*" is converted to an expression with type "pointer to *type*" that points to the initial element of the array object and is not an lvalue. If the array object has register storage class, the behavior is undefined.

4 A *function designator* is an expression that has function type. Except when it is the operand of the **typeof** specifier, the **sizeof** operator,[72] or the unary & operator, a function designator with type "function returning *type*" is converted to an expression that has type "pointer to function returning *type*".

5 Other than specified in the following, lambda types shall not be converted to any other object type. A complete function literal with a type "lambda with prototype `P`" can be converted implicitly or explicitly to an expression that has type "pointer to `Q`", where `Q` is a function type that is compatible with `P`.[73] For a type-generic function literal expression, types of underspecified parameters shall first be completed according to the parameters of the target prototype, that is, for each underspecified parameter there shall be a type specifier of a unique type as described in 6.7.11 such that the adjusted parameter type is the same as the adjusted parameter type of the target function type; after that, the prototype `P` of the thus completed lambda expression shall be the target prototype `Q`.[74] The function pointer value behaves as if a function `F` of type `P` with internal linkage, a unique name, and the same function body as for λ, where uses of identifiers from enclosing blocks in expressions that are not evaluated are replaced by proper types or values, had been defined in the translation unit and the function pointer had been formed by function-to-pointer conversion of that function. The only differences are that, if λ is not type-generic, the resulting function pointer is the same for the whole program execution whenever a conversion of λ is met[75] and that the function pointer needs not necessarily to be distinct from any other compatible function pointer that provides the same observable behavior.

Forward references: lambda expressions (6.5.2.6) address and indirection operators (6.5.3.2), assignment operators (6.5.16), common definitions `<stddef.h>` (7.19), **typeof** specifier 6.7.9, initialization (6.7.10), postfix increment and decrement operators (6.5.2.4), prefix increment and decrement operators (6.5.3.1), the **sizeof** and **_Alignof** operators (6.5.3.4), structure and union members (6.5.2.3)~~.~~, type inference (6.7.11).

[72] Because this conversion does not occur, the operand of the **sizeof** operator remains a function designator and violates the constraints in 6.5.3.4.

[73] It follows that lambdas of different type cannot be assigned to each other. Thus, in the conversion of a function literal to a function pointer, the prototype of the originating lambda expression can be assumed to be known, and a diagnostic can be issued if the prototypes do not aggree.

[74] Thus a specification of the target function pointer type in a conversion from a type-generic function literal expression that uses the `[*]` syntax for VM types is invalid.

[75] Thus a function literal that is not type-generic has properties that are similar to a function declared with **static** and **inline**. A possible implementation of the lambda type is to be the the function pointer type to which they convert.

— a type that is the signed or unsigned type corresponding to the effective type of the object,

— a type that is the signed or unsigned type corresponding to a qualified version of the effective type of the object,

— an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or

— a character type.

8 A floating expression may be *contracted*, that is, evaluated as though it were a single operation, thereby omitting rounding errors implied by the source code and the expression evaluation method.[100] The **FP_CONTRACT** pragma in <math.h> provides a way to disallow contracted expressions. Otherwise, whether and how expressions are contracted is implementation-defined.[101]

Forward references: the **FP_CONTRACT** pragma (7.12.2), copying functions (7.24.2).

6.5.1 Primary expressions

Syntax

1 *primary-expression:*
identifier
constant
string-literal
(*expression* **)**
generic-selection

Semantics

2 An identifier is a primary expression, provided it has been declared as designating an object (in which case it is an lvalue) or a function (in which case it is a function designator).[102]

3 A constant is a primary expression. Its type depends on its form and value, as detailed in 6.4.4.

4 A string literal is a primary expression. It is an lvalue with type as detailed in 6.4.5.

5 A parenthesized expression is a primary expression. Its type and value are identical to those of the unparenthesized expression. It is an lvalue, a function designator, a lambda expression, or a void expression if the unparenthesized expression is, respectively, an lvalue, a function designator, a lambda expression, or a void expression.

6 A generic selection is a primary expression. Its type and value depend on the selected generic association, as detailed in the following subclause.

Forward references: declarations (6.7).

6.5.1.1 Generic selection

Syntax

1 *generic-selection:*
_Generic (*assignment-expression* **,** *generic-assoc-list* **)**
generic-assoc-list:
generic-association
generic-assoc-list **,** *generic-association*
generic-association:
type-name **:** *assignment-expression*

[100] The intermediate operations in the contracted expression are evaluated as if to infinite range and precision, while the final operation is rounded to the format determined by the expression evaluation method. A contracted expression might also omit the raising of floating-point exceptions.

[101] This license is specifically intended to allow implementations to exploit fast machine instructions that combine multiple C operators. As contractions potentially undermine predictability, and can even decrease accuracy for containing expressions, their use needs to be well-defined and clearly documented.

[102] Thus, an undeclared identifier is a violation of the syntax.

`default :` *assignment-expression*

Constraints

2 A generic selection shall have no more than one **`default`** generic association. The type name in a generic association shall specify a complete object type other than a variably modified type. No two generic associations in the same generic selection shall specify compatible types. The type of the controlling expression is the type of the expression as if it had undergone an lvalue conversion,[103)] array to pointer conversion, or function to pointer conversion. That type shall be compatible with at most one of the types named in the generic association list. If a generic selection has no **`default`** generic association, its controlling expression shall have type compatible with exactly one of the types named in its generic association list.

Semantics

3 The controlling expression of a generic selection is not evaluated. If a generic selection has a generic association with a type name that is compatible with the type of the controlling expression, then the result expression of the generic selection is the expression in that generic association. Otherwise, the result expression of the generic selection is the expression in the **`default`** generic association. None of the expressions from any other generic association of the generic selection is evaluated.

4 The type and value of a generic selection are identical to those of its result expression. It is an lvalue, a function designator, a lambda expression, or a void expression if its result expression is, respectively, an lvalue, a function designator, a lambda expression, or a void expression. A generic selection that is the operand of a **`typeof`** specification behaves as if the selected assignment expression had been the operand.

5 **EXAMPLE** The **`cbrt`** type-generic macro could be implemented as follows:

```
#define cbrt(X) _Generic((X),                   \
                         long double: cbrtl,    \
                         default: cbrt,         \
                         float: cbrtf           \
                         )(X)
```

6.5.2 Postfix operators

Syntax

1 *postfix-expression:*
primary-expression
postfix-expression **`[`** *expression* **`]`**
postfix-expression **`(`** *argument-expression-list*$_{\text{opt}}$ **`)`**
postfix-expression **`.`** *identifier*
postfix-expression **`->`** *identifier*
postfix-expression **`++`**
postfix-expression **`-`**
`(` *type-name* **`)`** **`{`** *initializer-list* **`}`**
`(` *type-name* **`)`** **`{`** *initializer-list* **`,`** **`}`**
lambda-expression

argument-expression-list:
assignment-expression
argument-expression-list **`,`** *assignment-expression*

[103)] An lvalue conversion drops type qualifiers.

6.5.2.1 Array subscripting

Constraints

1 One of the expressions shall have type "pointer to complete object *type*", the other expression shall have integer type, and the result has type "*type*".

Semantics

2 A postfix expression followed by an expression in square brackets `[]` is a subscripted designation of an element of an array object. The definition of the subscript operator `[]` is that `E1[E2]` is identical to `(*((E1)+(E2)))`. Because of the conversion rules that apply to the binary `+` operator, if `E1` is an array object (equivalently, a pointer to the initial element of an array object) and `E2` is an integer, `E1[E2]` designates the `E2` -th element of `E1` (counting from zero).

3 Successive subscript operators designate an element of a multidimensional array object. If `E` is an n-dimensional array ($n \geq 2$) with dimensions $i \times j \times \cdots \times k$, then `E` (used as other than an lvalue) is converted to a pointer to an $(n-1)$-dimensional array with dimensions $j \times \cdots \times k$. If the unary $*$ operator is applied to this pointer explicitly, or implicitly as a result of subscripting, the result is the referenced $(n-1)$-dimensional array, which itself is converted into a pointer if used as other than an lvalue. It follows from this that arrays are stored in row-major order (last subscript varies fastest).

4 **EXAMPLE** Consider the array object defined by the declaration

```
int x[3][5];
```

Here `x`

is a 3×5 array of

`int` s; more precisely, `x` is an array of three element objects, each of which is an array of five **`int`** s. In the expression `x[i]`, which is equivalent to `(*((x)+(i)))`, `x` is first converted to a pointer to the initial array of five **`int`** s. Then `i` is adjusted according to the type of `x`, which conceptually entails multiplying `i` by the size of the object to which the pointer points, namely an array of five **`int`** objects. The results are added and indirection is applied to yield an array of five **`int`** s. When used in the expression `x[i][j]`, that array is in turn converted to a pointer to the first of the **`int`** s, so `x[i][j]` yields an **`int`**.

Forward references: additive operators (6.5.6), address and indirection operators (6.5.3.2), array declarators (6.7.6.2).

6.5.2.2 Function calls

Constraints

1 The ~~expression that denotes the called function~~postfix expression[104] shall have ~~type~~ lambda type or pointer to function type, returning **`void`** or returning a complete object type other than an array type.

2 If the ~~expression that denotes the called function has a type that~~ postfix expression is a lambda or if the type of the function includes a prototype, the number of arguments shall agree with the number of parameters of the function or lambda type. Each argument shall have a type such that its value may be assigned to an object with the unqualified version of the type of its corresponding parameter.

Semantics

3 A postfix expression followed by parentheses `()` containing a possibly empty, comma-separated list of expressions is a function call. The postfix expression denotes the called function or lambda. The list of expressions specifies the arguments to the function or lambda.

4 An argument may be an expression of any complete object type. In preparing for the call to a function, the arguments are evaluated, and each parameter is assigned the value of the corresponding argument.[105]

5 If the expression that denotes the called function has lambda type or type pointer to function returning an object type, the function call expression has the same type as that object type, and has the value determined as specified in 6.8.6.4. Otherwise, the function call has type **`void`**.

[104] Most often, this is the result of converting an identifier that is a function designator.

[105] A function or lambda can change the values of its parameters, but these changes cannot affect the values of the arguments. On the other hand, it is possible to pass a pointer to an object, and the function or lambda can then change the value of the object pointed to. A parameter declared to have array or function type is adjusted to have a pointer type as described in 6.9.1.

6 If the expression that denotes the called function has a type that does not include a prototype, the integer promotions are performed on each argument, and arguments that have type **`float`** are promoted to **`double`**. These are called the *default argument promotions*. If the number of arguments does not equal the number of parameters, the behavior is undefined. If the function is defined with a type that includes a prototype, and either the prototype ends with an ellipsis (`, ...`) or the types of the arguments after promotion are not compatible with the types of the parameters, the behavior is undefined. If the function is defined with a type that does not include a prototype, and the types of the arguments after promotion are not compatible with those of the parameters after promotion, the behavior is undefined, except for the following cases:

— one promoted type is a signed integer type, the other promoted type is the corresponding unsigned integer type, and the value is representable in both types;

— both types are pointers to qualified or unqualified versions of a character type or **`void`**.

7 If the expression that denotes the called function is a lambda or is a function has a type that does include a prototype, the arguments are implicitly converted, as if by assignment, to the types of the corresponding parameters, taking the type of each parameter to be the unqualified version of its declared type. The ellipsis notation in a function prototype declarator causes argument type conversion to stop after the last declared parameter. The default argument promotions are performed on trailing arguments.

8 No other conversions are performed implicitly; in particular, the number and types of arguments are not compared with those of the parameters in a function definition that does not include a function prototype declarator.

9 If the lambda or function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called lambda or function, the behavior is undefined.

10 There is a sequence point after the evaluations of the function designator and the actual arguments but before the actual call. Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or after the execution of the body of the called function or lambda is indeterminately sequenced with respect to the execution of the called function.[106)]

11 Recursive function calls shall be permitted, both directly and indirectly through any chain of other functions or lambdas.

12 **EXAMPLE** In the function call

```
(*pf[f1()]) (f2(), f3() + f4())
```

the functions `f1`, `f2`, `f3`, and `f4` can be called in any order. All side effects have to be completed before the function pointed to by `pf[f1()]` is called.

Forward references: function declarators (including prototypes) (6.7.6.3), function definitions (6.9.1), the **`return`** statement (6.8.6.4), simple assignment (6.5.16.1).

6.5.2.3 Structure and union members

Constraints

1 The first operand of the `.` operator shall have an atomic, qualified, or unqualified structure or union type, and the second operand shall name a member of that type.

2 The first operand of the`->` operator shall have type “pointer to atomic, qualified, or unqualified structure” or “pointer to atomic, qualified, or unqualified union”, and the second operand shall name a member of the type pointed to.

Semantics

3 A postfix expression followed by the `.` operator and an identifier designates a member of a structure or union object. The value is that of the named member,[107)] and is an lvalue if the first expression is

[106)] In other words, function executions do not “interleave” with each other.

[107)] If the member used to read the contents of a union object is not the same as the member last used to store a value in the object, the appropriate part of the object representation of the value is reinterpreted as an object representation in the new

The first always has static storage duration and has type array of **char**, but need not be modifiable; the last two have automatic storage duration when they occur within the body of a function, and the first of these two is modifiable.

13 **EXAMPLE 6** Like string literals, const-qualified compound literals can be placed into read-only memory and can even be shared. For example,

```
(const char []){"abc"} == "abc"
```

might yield 1 if the literals' storage is shared.

14 **EXAMPLE 7** Since compound literals are unnamed, a single compound literal cannot specify a circularly linked object. For example, there is no way to write a self-referential compound literal that could be used as the function argument in place of the named object `endless_zeros` below:

```
struct int_list { int car; struct int_list *cdr; };
struct int_list endless_zeros = {0, &endless_zeros};
eval(endless_zeros);
```

15 **EXAMPLE 8** Each compound literal creates only a single object in a given scope:

```
struct s { int i; };

int f (void)
{
      struct s *p = 0, *q;
      int j = 0;

again:
      q = p, p = &((struct s){ j++ });
      if (j  <  2) goto again;

      return p == q && q->i == 1;
}
```

The function `f()` always returns the value 1.

16 Note that if an iteration statement were used instead of an explicit **goto** and a labeled statement, the lifetime of the unnamed object would be the body of the loop only, and on entry next time around `p` would have an indeterminate value, which would result in undefined behavior.

Forward references: type names (6.7.7), initialization (6.7.10).

6.5.2.6 Lambda expressions

Syntax

1 *lambda-expression:*
capture-clause *parameter-clause*$_{\text{opt}}$ *attribute-specifier-sequence*$_{\text{opt}}$ *function-body*

capture-clause:
[*capture-list*$_{\text{opt}}$ **]**

capture-list:
capture-list-element
capture-list **,** *capture-list-element*

capture-list-element:
value-capture
identifier-capture

value-capture:
capture **=** *assignment-expression*

identifier-capture:
& *capture*

capture:
identifier

parameter-clause:
(*parameter-list*$_{\text{opt}}$ **)**

Constraints

2 An identifier shall appear at most once; either as a capture or as a parameter name in the parameter list. The identifier of an identifier capture shall designate an object of automatic storage duration that is defined in a scope that surrounds the lambda expression.

3 Within the lambda expression, identifiers (including captures and parameters of the lambda) shall be used according to the usual scoping rules, but outside the assignment expression of a value capture the following exceptions apply to identifiers that are declared in a block that strictly encloses the lambda expression and that are not identifier captures:

— Objects or type definitions with variably modified type shall not be used.

— Objects with automatic storage duration shall not be evaluated.[114)]

4 After determining the type of all captures and parameters, either directly or because a type-generic lambda appears in a function-call or conversion to function pointer, the function body shall be such that a return type *type* according to the rules in 6.8.6.4 can be inferred. If the lambda occurs in a conversion to a function pointer, the inferred return type shall be compatible to the specified return type of the function pointer; if additionally the lambda is type-generic, the return type shall be the same as the specified return type.

5 When a lambda expression with an underspecified parameter is evaluated as a void expression, the capture clause shall fulfill the constraints as specified above. The parenthesized parameter list shall provide a valid list of declarations of parameters, only that one or more of these may have an underspecified type. After that shall follow a `{` token, a balanced token sequence (**??**), and a `}` token.[115)]

Semantics

6 The optional attribute specifier sequence in a lambda expression appertains to the resulting lambda type and to its function prototype. If the parameter clause is omitted, a clause of the form `()` is assumed. A lambda expression without any capture is called a *function literal expression* , otherwise it is called a *closure expression* . A lambda value originating from a function literal expression is called a *function literal* , otherwise it is called a *closure* .

7 Similar to a function definition, a lambda expression forms a single block that comprises all of its parts. Each capture and parameter has a scope of visibility that starts immediately after its definition is completed and extends to the end of the function body. In particular, captures and parameters are visible throughout the whole function body, unless they are redeclared in a depending block within that function body. Value captures and parameters have automatic storage duration; in each function call to the formed lambda value, a new instance of each value capture and parameter is created and initialized in order of declaration and has a lifetime until the end of the call, only that the addresses of value captures are not necessarily unique.

[114)] Identifiers of visible automatic objects that are not captures and that do not have a VM type, may still be used if they are not evaluated, for example in **`sizeof`** expressions, in **`typeof`** specifiers (if they are not lambdas themselves) or as controlling expression of a generic primary expression.

[115)] That means, besides the validity of the capture clause and the parameter list, an implementation is only required to parse the function body as a token sequence but is not required to diagnose additional constraints, such as the validity of the use of keywords or identifiers within the function body if these are possibly restricted through a syntax derivation or additional constraints.

8 A lambda expression for which at least one parameter declaration in the parameter list has no type specifier is a *type-generic lambda* with an imcomplete lambda type. It shall only be evaluated as a void expression, be the postfix expression of a function call or, if the capture clause is empty, be the operand of a conversion to a pointer to function with fully specified parameter types, see 6.3.2.1. For a void expression, it has only the side effects that result from the evaluation of the capture clause and shall be syntactically correct as indicated in the constraints; the translation may fail, if the function body is such that no possible function call arguments or target types for a conversion could successfully complete the lambda type; the lambda expression shall otherwise be ignored.

9 For a function call, the type of an argument (after lvalue, array-to-pointer or function-to-pointer conversion) to an underspecified parameter shall be such that it can be used to complete the type of that parameter analogous to 6.7.11, only that the inferred type for an parameter of array or function type is adjusted analogously to function declarators (6.7.6.3) to a possibly qualified object pointer type (for an array) or to a function pointer type (for a function) to match type of the argument. For a conversion of any arguments, the parameter types shall be those of the function type.

10 The assignment expression E in the definition of a value capture determines a value E_0 with type T_0, which is E after possible lvalue, array-to-pointer or function-to-pointer conversion. The type of the capture is T_0 **const** and its value is E_0 for all evaluations in all function calls to the lambda value. If, within the function body, the address of the capture or one of its members is taken, either explicitly by applying a unary & operator or by an array to pointer conversion,[116)] and that address is used to modify the underlying object, the behavior is undefined.

11 The evaluation of the assignment expressions of value captures takes place during each evaluation of the lambda expression. The evaluations for the value captures are sequenced in order of declaration; an earlier capture may occur within an assignment expression of a later one. The evaluation of a lambda expression is sequenced before any use of the resulting lambda value. For each call to a lambda value, value captures (with type and value as determined during the evaluation of the lambda expression) and then parameter types and values are determined in order of declaration. Value captures and earlier parameters may occur within the declaration of a later one.

12 The object of automatic storage duration of the surrounding scope that corresponds to an identifier capture shall be visible within the function body according to the usual scoping rules and shall be accessible within the function body throughout each call to the lambda. If the definition of the object uses the **register** storage class, the behavior is undefined. Access to the object within a call to the lambda follows the happens-before relation, in particular modifications to the object that happen before the call are visible within the call, and modifications to the object within the call are visible for all evaluations that happen after the call.[117)]

13 For each lambda expression, the return type *type* is inferred as indicated in the constraints. A lambda expression λ that is not type-generic has an unspecified lambda type L that is the same for every evaluation of λ; as a result of the expression, a value of type L is formed that identifies λ and the specific set of values of the identifiers in the capture clause for the evaluation, if any. This is called a *lambda value* . It is unspecified, whether two lambda expressions λ and κ share the same lambda type even if they are lexically equal but appear at different points of the program. Objects of lambda type shall not be modified other then by simple assignment.

14 A lambda expression λ that is generic has an incomplete lambda type that is completed when the expression is used directly in a function call expression or converted to a function pointer. When used in a function call, the parameter types are inferred in order of declaration, but after the evaluation of the assignment expressions of the explicit value captures, after which the return type of the lambda is inferred from the function body. The so completed lambda value is then used in the function call which is sequenced after the evaluation of the lambda expression.

15 **NOTE 1** A direct function call to a function literal expression can be modeled by first performing a conversion of the function literal to a function pointer and then calling that function pointer.

[116)] The capture does not have array type, but if it has a union or structure type, one of its members may have such a type.

[117)] That is, evaluation of the identifier results in the same lvalue with the same type and address as for the scope surrounding the lambda. In particular, it is possible that the value of such an object becomes indeterminate after a call to **longjmp**, see 7.13.2.1.

16 **NOTE 2** A direct function call to a closure expression with parameters (possibly type-generic)

```
[ captures ] (decl1, ..., decln) {
  block-item-list
}(arg1, ..., argn)
```

can be modeled with a such a call to a closure expression without parameters

```
[ _ArgCap1 = arg1, ..., _ArgCapn = argn, captures ] (void) {
  decl1 = _ArgCap1;
  ...
  decln = _ArgCapn;
  block-item-list
}()
```

where $\texttt{\_ArgCap}_1, \ldots, \texttt{\_ArgCap}_n$ are new identifiers that are unique for the translation unit. This equivalence uses the fact that the evaluation of the argument expressions $\texttt{arg}_1, \ldots, \texttt{arg}_n$ and the original closure expression as a whole can be evaluated without sequencing constraints before the actual function call operation. In particular, side effects that occur during the evaluation of any of the arguments or the capture list will not effect one another. This not withstanding, side effects that have an influence about the evaluation of captures in the specified capture list or that determine the type of parameters occur sequenced as specified in the original closure expression.

Recommended practice

17 Implementations are encouraged to diagnose any attempt to modify a lambda type object other than by assignment.

18 **EXAMPLE 1** The usual scoping rules extend to lambda expressions; the concept of captures only restricts which identifiers may be evaluated or not.

```
#include <stdio.h>
static long v;
int main(void) {
  [     ](void){ printf("%ld\n", v); }();               // valid, prints 0
  [     ](void){ printf("%zu\n", sizeof v); }();        // valid, prints sizeof(long)
  int v = 5;
  [     ](void){ printf("%d\n", v); }();                // invalid
  [     ](void){ extern long v; printf("%ld\n", v; }(); // valid, prints 0

  auto const λ = [v = v](void){ printf("%d\n", v); };  // freeze and shadow v
  [&v   ](void){ v = 7; printf("%d\n", v); }();         // valid, prints 7
  λ();                                                 // valid, prints 5
  [v = v](void){ printf("%d\n", v); }();               // valid, prints 7
  [v = v](void){ printf("%zu\n", sizeof v); }();       // valid, prints sizeof(int)
  [     ](void){ printf("%zu\n", sizeof v); }();       // valid, prints sizeof(int)

}
```

19 **EXAMPLE 2** The following uses a function literal as a comparison function argument for **qsort**.

```
#define SORTFUNC(TYPE) [](size_t nmemb, TYPE A[nmemb]) {                \
  qsort(A, nmemb, sizeof(A[0]),                                         \
        [](void const* x, void const* y){          /* comparison lambda  */ \
          TYPE X = *(TYPE const*)x;                                     \
          TYPE Y = *(TYPE const*)y;                                     \
          return (X < Y) ? -1 : ((X > Y) ? 1 : 0); /* return of type int */ \
        }                                                               \
        );                                                              \
  return A;                                                             \
  }
  ...
  long C[5] = { 4, 3, 2, 1, 0, };
  SORTFUNC(long)(5, C);                        // lambda → (pointer →) function call
```

```
...
auto const sortDouble = SORTFUNC(double);    // lambda value → lambda object
double* (*sF)(size_t nmemb, double[nmemb]) = sortDouble;   // conversion
...
double* ap = sortDouble(4, (double[]){ 5, 8.9, 0.1, 99, });
double B[27] = { /* some values ... */ };
sF(27, B);                                   // reuses the same function
...
double* (*sG)(size_t nmemb, double[nmemb]) = SORTFUNC(double); // conversion
```

This code evaluates the macro `SORTFUNC` twice, therefore in total four lambda expressions are formed.

The function literals of the "comparison lambdas" are not operands of a function call expression, and so by conversion a pointer to function is formed and passed to the corresponding call of **`qsort`**. Since the respective captures are empty, the effect is as if to define two comparison functions, that could equally well be implemented as **`static`** functions with auxiliary names and these names could be used to pass the function pointers to **`qsort`**.

The outer lambdas are again without capture. In the first case, for **`long`**, the lambda value is subject to a function call, and it is unspecified if the function call uses a specific lambda type or directly uses a function pointer. For the second, a copy of the lambda value is stored in the variable `sortDouble` and then converted to a function pointer `sF`. Other than for the difference in the function arguments, the effect of calling the lambda value (for the compound literal) or the function pointer (for array `B`) is the same.

For optimization purposes, an implementation may fold lambda values that are expanded at different points of the program such that effectively only one function is generated. For example here the function pointers `sF` and `sG` may or may not be equal.

20 **EXAMPLE 3** Consider the following type-generic function literal that computes the maximum value of two parameters `X` and `Y`.

```
#define MAXIMUM(X, Y)                                \
   [](auto a, auto b){                               \
     return (a < 0)                                  \
            ? ((b <  0) ? ((a < b) ? b : a) : b)     \
            : ((b >= 0) ? ((a < b) ? b : a) : a);    \
   }(X , Y)

   auto R = MAXIMUM(-1, -1U);
   auto S = MAXIMUM(-1U, -1L);
```

After preprocessing, the definition of `R`, becomes

```
auto R = [](auto a, auto b){
  return (a < 0)
     ? ((b <  0) ? ((a < b) ? b : a) : b)
     : ((b >= 0) ? ((a < b) ? b : a) : a);
  }(-1, -1U);
```

To determine type and value of `R`, first the type of the parameters in the function call are inferred to be **`signed int`** and **`unsigned int`**, respectively. With this information, the type of the **`return`** expression becomes the common arithmetic type of the two, which is **`unsigned int`**. Thus the return type of the lambda is that type. The resulting lambda value is the first operand to the function call operator `()`. So `R` has the type **`unsigned int`** and a value of **`UINT_MAX`**.

For `S`, a similar deduction shows that the value still is **`UINT_MAX`** but the type could be **`unsigned int`** (if **`int`** and **`long`** have the same width) or **`long`** (if **`long`** is wider than **`int`**).

As long as they are integers, regardless of the specific type of the arguments, the type of the expression is always such that the mathematical maximum of the values fits. So `MAXIMUM` implements a type-generic maximum macro that is suitable for any combination of integer types.

21 **EXAMPLE 4**

```
void matmult(size_t k, size_t n, size_t m,
             double const A[k][n], double const B[n][m], double const C[k][m]) {
  // dot product with stride of m for B
  // ensure constant propagation of n and m
  auto const λ0 = [ν=n, μ=m](double const x[ν], double const B[ν][μ], size_t m0) {
    double ret = 0.0;
```

```
    for (size_t i = 0; i < ν; ++i) {
      ret += x[i]*B[i][m0];
    }
    return ret;
  };
  // vector matrix product
  // ensure constant propagation of n and m, and accessibility of λ0
  auto const λ1 = [ν=n, μ=m, &λ0](double const x[ν], double const B[ν][μ],
                                  double res[m]) {
    for (size_t m0 = 0; m0 < μ; ++m0) {
      res[m0] = λ0(x, B, m0);
    }
  };
  for (size_t k0 = 0; k0 < k; ++k0) {
    double const (*Ap)[l] = A[k0];
    double (*Cp)[m] = C[k0];
    λ1(*Ap, B, *Cp);
  }
}
```

This function evaluates two closures; `λ0` has a return type of **`double`**, `λ1` of **`void`**. Both lambda values serve repeatedly as first operand to function evaluation but the evaluation of the captures is only done once for each of the closures. For the purpose of optimization, an implementation could generate copies of the underlying functions for each evaluation of such a closure such that the values of the captures ν and μ are replaced on a machine instruction level.

6.5.3 Unary operators

Syntax

1 *unary-expression:*

postfix-expression
++ *unary-expression*
- *unary-expression*
unary-operator cast-expression
sizeof *unary-expression*
sizeof **(** *type-name* **)**
_Alignof **(** *type-name* **)**

unary-operator: one of

& * + - ~ !

6.5.3.1 Prefix increment and decrement operators

Constraints

1 The operand of the prefix increment or decrement operator shall have atomic, qualified, or unqualified real or pointer type, and shall be a modifiable lvalue.

Semantics

2 The value of the operand of the prefix `++` operator is incremented. The result is the new value of the operand after incrementation. The expression `++E` is equivalent to `(E+=1)`. See the discussions of additive operators and compound assignment for information on constraints, types, side effects, and conversions and the effects of operations on pointers.

3 The prefix `--` operator is analogous to the prefix `++` operator, except that the value of the operand is decremented.

Forward references: additive operators (6.5.6), compound assignment (6.5.16.2).

Constraints

2 Unless the type name specifies a void type, the type name shall specify atomic, qualified, or unqualified scalar type, and the operand shall have scalar type, or, the type name shall specify an atomic, qualified, or unqualified pointer to function with prototype, and the operand is a function literal such that a conversion (6.3.2.1) from the function literal to the function pointer type is defined.

3 Conversions that involve pointers, other than where permitted by the constraints of 6.5.16.1, shall be specified by means of an explicit cast.

4 A pointer type shall not be converted to any floating type. A floating type shall not be converted to any pointer type.

Semantics

5 Preceding an expression by a parenthesized type name converts the value of the expression to the unqualified version of the named type. This construction is called a *cast*.[120] A cast that specifies no conversion has no effect on the type or value of an expression.

6 If the value of the expression is represented with greater range or precision than required by the type named by the cast (6.3.1.8), then the cast specifies a conversion even if the type of the expression is the same as the named type and removes any extra range and precision.

Forward references: equality operators (6.5.9), function declarators (including prototypes) (6.7.6.3), simple assignment (6.5.16.1), type names (6.7.7).

6.5.5 Multiplicative operators

Syntax

1 *multiplicative-expression:*
cast-expression
multiplicative-expression **`*`** *cast-expression*
multiplicative-expression **`/`** *cast-expression*
multiplicative-expression **`%`** *cast-expression*

Constraints

2 Each of the operands shall have arithmetic type. The operands of the `%` operator shall have integer type.

Semantics

3 The usual arithmetic conversions are performed on the operands.

4 The result of the binary `*` operator is the product of the operands.

5 The result of the `/` operator is the quotient from the division of the first operand by the second; the result of the `%` operator is the remainder. In both operations, if the value of the second operand is zero, the behavior is undefined.

6 When integers are divided, the result of the `/` operator is the algebraic quotient with any fractional part discarded.[121] If the quotient `a/b` is representable, the expression `(a/b)*b + a%b` shall equal `a`; otherwise, the behavior of both `a/b` and `a%b` is undefined.

6.5.6 Additive operators

Syntax

1 *additive-expression:*
multiplicative-expression
additive-expression **`+`** *multiplicative-expression*
additive-expression **`-`** *multiplicative-expression*

[120] A cast does not yield an lvalue.
[121] This is often called "truncation toward zero".

Constraints

2 An assignment operator shall have a modifiable lvalue as its left operand.

Semantics

3 An assignment operator stores a value in the object designated by the left operand. An assignment expression has the value of the left operand after the assignment,[127)] but is not an lvalue. The type of an assignment expression is the type the left operand would have after lvalue conversion. The side effect of updating the stored value of the left operand is sequenced after the value computations of the left and right operands. The evaluations of the operands are unsequenced.

6.5.16.1 Simple assignment

Constraints

1 One of the following shall hold:[128)]

— the left operand has atomic, qualified, or unqualified arithmetic type, and the right has arithmetic type;

— the left operand has an atomic, qualified, or unqualified version of a structure or union type compatible with the type of the right;

— the left operand has the unqualified version of the lambda type of the right;

— the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) both operands are pointers to qualified or unqualified versions of compatible types, and the type pointed to by the left has all the qualifiers of the type pointed to by the right;

— the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) one operand is a pointer to an object type, and the other is a pointer to a qualified or unqualified version of **`void`**, and the type pointed to by the left has all the qualifiers of the type pointed to by the right;

— the left operand is an atomic, qualified, or unqualified pointer to function with a prototype, the right operand is a function literal, and the prototypes of the function pointer and of the function literal shall be such that a conversion from the function literal to the function pointer type is defined;

— the left operand is an atomic, qualified, or unqualified pointer, and the right is a null pointer constant; or

— the left operand has type atomic, qualified, or unqualified **`_Bool`**, and the right is a pointer.

Semantics

2 In *simple assignment* (**=**), the value of the right operand is converted to the type of the assignment expression and replaces the value stored in the object designated by the left operand.

3 If the value being stored in an object is read from another object that overlaps in any way the storage of the first object, then the overlap shall be exact and the two objects shall have qualified or unqualified versions of a compatible type; otherwise, the behavior is undefined.

4 **EXAMPLE 1** In the program fragment

```
int f(void);
char c;
/* ... */
```

[127)] The implementation is permitted to read the object to determine the value but is not required to, even when the object has volatile-qualified type.

[128)] The asymmetric appearance of these constraints with respect to type qualifiers is due to the conversion (specified in 6.3.2.1) that changes lvalues to "the value of the expression" and thus removes any type qualifiers that were applied to the type category of the expression (for example, it removes **`const`** but not **`volatile`** from the type **`int volatile * const`**).

```
if ((c = f()) == -1)
      /* ... */
```

the **int** value returned by the function could be truncated when stored in the **char**, and then converted back to **int** width prior to the comparison. In an implementation in which "plain" **char** has the same range of values as **unsigned char** (and **char** is narrower than **int**), the result of the conversion cannot be negative, so the operands of the comparison can never compare equal. Therefore, for full portability, the variable c would be declared as **int**.

5 **EXAMPLE 2** In the fragment:

```
char c;
int i;
long l;

l = (c = i);
```

the value of i is converted to the type of the assignment expression c = i, that is, **char** type. The value of the expression enclosed in parentheses is then converted to the type of the outer assignment expression, that is, **long int** type.

6 **EXAMPLE 3** Consider the fragment:

```
const char **cpp;
char *p;
const char c = 'A';

cpp = &p;       // constraint violation
*cpp = &c;      // valid
*p = 0;         // valid
```

The first assignment is unsafe because it would allow the following valid code to attempt to change the value of the const object c.

7 **EXAMPLE 4** Lambda types can be assigned in a portable way, only if both lambda types originate from the same lambda expression.

```
auto λ = [s = 0](){ puts("hello"); };
auto κ = [s = 0](){ puts("hello"); };
κ = λ;                                    // invalid, different types
auto λp = (false ? &λ : malloc(sizeof(λ))); // pointer to lambda
*λp = λ;                                  // valid, same type
(*λp)();                                  // valid, prints ``hello''
```

6.5.16.2 Compound assignment

Constraints

1 For the operators += and -= only, either the left operand shall be an atomic, qualified, or unqualified pointer to a complete object type, and the right shall have integer type; or the left operand shall have atomic, qualified, or unqualified arithmetic type, and the right shall have arithmetic type.

2 For the other operators, the left operand shall have atomic, qualified, or unqualified arithmetic type, and (considering the type the left operand would have after lvalue conversion) each operand shall have arithmetic type consistent with those allowed by the corresponding binary operator.

Semantics

3 A *compound assignment* of the form E1 *op*= E2 is equivalent to the simple assignment expression E1 = E1 *op* (E2), except that the lvalue E1 is evaluated only once, and with respect to an indeterminately-sequenced function call, the operation of a compound assignment is a single evaluation. If E1 has an atomic type, compound assignment is a read-modify-write operation with **memory_order_seq_cst** memory order semantics.

4 **NOTE** Where a pointer to an atomic object can be formed and E1 and E2 have integer type, this is equivalent to the following code sequence where *T1* is the type of E1 and *T2* is the type of E2:

```
T1 *addr = &E1;
```

6.7.1 Storage-class specifiers

Syntax

1 *storage-class-specifier:*

typedef
extern
static
_Thread_local
auto
register

Constraints

2 At most, one storage-class specifier may be given in the declaration specifiers in a declaration, except that **_Thread_local** may appear with **static** or **extern** , and that **auto** may appear with all others but with **typedef** .[135)]

3 In the declaration of an object with block scope, if the declaration specifiers include **_Thread_local**, they shall also include either **static** or **extern**. If **_Thread_local** appears in any declaration of an object, it shall be present in every declaration of that object.

4 **_Thread_local** shall not appear in the declaration specifiers of a function declaration. **auto** shall only appear in the declaration specifiers of an identifier with file scope if the declaration is also a definition or if a definition of that identifier is already visible.

Semantics

5 The **typedef** specifier is called a "storage-class specifier" for syntactic convenience only; it is discussed in 6.7.8. The meanings of the various linkages and storage durations were discussed in 6.2.2 and 6.2.4.

6 A declaration of an identifier for an object with storage-class specifier **register** suggests that access to the object be as fast as possible. The extent to which such suggestions are effective is implementation-defined.[136)]

7 The declaration of an identifier for a function that has block scope shall have no explicit storage-class specifier other than **extern**.

8 If an aggregate or union object is declared with a storage-class specifier other than **typedef**, the properties resulting from the storage-class specifier, except with respect to linkage, also apply to the members of the object, and so on recursively for any aggregate or union member objects.

9 If **auto** appears with another storage-class specifier, or if it appears in a declaration at file scope it is ignored for the purpose of determining a storage class or linkage. It then only indicates that the declared type may be inferred from an initializer (for objects see 6.7.11), or from the function body (for functions see 6.8.6.4).

Forward references: type definitions (6.7.8), type inference (6.7.11), function definitions (6.9.1).

6.7.2 Type specifiers

Syntax

1 *type-specifier:*

void
char
short
int

[135)] See "future language directions" (6.11.5).

[136)] The implementation can treat any **register** declaration simply as an **auto** declaration. However, whether or not addressable storage is actually used, the address of any part of an object declared with storage-class specifier **register** cannot be computed, either explicitly (by use of the unary & operator as discussed in 6.5.3.2) or implicitly (by converting an array name to a pointer as discussed in 6.3.2.1). Thus, the only operator that can be applied to an array declared with storage-class specifier **register** is **sizeof**.

long
float
double
signed
unsigned
_Bool
_Complex
atomic-type-specifier
struct-or-union-specifier
enum-specifier
typedef-name
typeof-specifier

Constraints

2 ~~At~~ Except where the type is inferred (6.7.11, 6.8.6.4, 6.9.1), at least one type specifier shall be given in the declaration specifiers in each declaration, and in the specifier-qualifier list in each struct declaration and type name. Each list of type specifiers shall be one of the following multisets (delimited by commas, when there is more than one multiset per item); the type specifiers may occur in any order, possibly intermixed with the other declaration specifiers.

— **void**
— **char**
— **signed char**
— **unsigned char**
— **short**, **signed short**, **short int**, or **signed short int**
— **unsigned short**, or **unsigned short int**
— **int**, **signed**, or **signed int**
— **unsigned**, or **unsigned int**
— **long**, **signed long**, **long int**, or **signed long int**
— **unsigned long**, or **unsigned long int**
— **long long**, **signed long long**, **long long int**, or **signed long long int**
— **unsigned long long**, or **unsigned long long int**
— **float**
— **double**
— **long double**
— **_Bool**
— **float _Complex**
— **double _Complex**
— **long double _Complex**
— atomic type specifier
— ~~struct or union~~ **struct** or **union** specifier
— ~~enum~~ **enum** specifier
— ~~typedef name~~ **typedef** name
— **typeof** specifier.

3 The type specifier **_Complex** shall not be used if the implementation does not support complex types (see 6.10.8.3).

Semantics

4 Specifiers for structures, unions, enumerations, and atomic types are discussed in 6.7.2.1 through 6.7.2.4. Declarations of typedef names are discussed in 6.7.8. The characteristics of the other types are discussed in 6.2.5. Declarations for which the type specifiers are inferred from initializers are discussed in 6.7.11.

5 Each of the comma-separated multisets designates the same type, except that for bit-fields, it is implementation-defined whether the specifier **`int`** designates the same type as **`signed int`** or the same type as **`unsigned int`**.

6 A declaration that contains no type specifier is said to be *underspecified*. Identifiers that are such declared have incomplete type. Their type can be completed by type inference from an initialization (for objects) or from **`return`** statements in a function body (for return types of functions).

Forward references: the **`return`** statement (6.8.6.4), atomic type specifiers (6.7.2.4), enumeration specifiers (6.7.2.2), structure and union specifiers (6.7.2.1), tags (6.7.2.3), type definitions (6.7.8), type inference (6.7.11).

6.7.2.1 Structure and union specifiers

Syntax

1 *struct-or-union-specifier:*
struct-or-union *identifier*$_{\text{opt}}$ **`{`** *struct-declaration-list* **`}`**
struct-or-union *identifier*

struct-or-union:
`struct`
`union`

struct-declaration-list:
struct-declaration
struct-declaration-list *struct-declaration*

struct-declaration:
specifier-qualifier-list *struct-declarator-list*$_{\text{opt}}$ **`;`**
static_assert-declaration

specifier-qualifier-list:
type-specifier *specifier-qualifier-list*$_{\text{opt}}$
type-qualifier *specifier-qualifier-list*$_{\text{opt}}$
alignment-specifier *specifier-qualifier-list*$_{\text{opt}}$

struct-declarator-list:
struct-declarator
struct-declarator-list **`,`** *struct-declarator*

struct-declarator:
declarator
declarator$_{\text{opt}}$ **`:`** *constant-expression*

Constraints

2 A struct-declaration that does not declare an anonymous structure or anonymous union shall contain a struct-declarator-list.

3 A structure or union shall not contain a member with incomplete or function type (hence, a structure shall not contain an instance of itself, but may contain a pointer to an instance of itself), except that the last member of a structure with more than one named member may have incomplete array type; such a structure (and any union containing, possibly recursively, a member that is such a structure) shall not be a member of a structure or an element of an array.

4 The expression that specifies the width of a bit-field shall be an integer constant expression with a nonnegative value that does not exceed the width of an object of the type that would be specified were the colon and expression omitted.[137] If the value is zero, the declaration shall have no

[137] While the number of bits in a **`_Bool`** object is at least **`CHAR_BIT`**, the width (number of sign and value bits) of a **`_Bool`** can

pointer:
`*` *type-qualifier-list*$_{\text{opt}}$ *pointer*
type-qualifier-list:
type-qualifier
type-qualifier-list type-qualifier
parameter-type-list:
parameter-list
parameter-list `, ...`
parameter-list:
parameter-declaration
parameter-list `,` *parameter-declaration*
parameter-declaration:
declaration-specifiers declarator
declaration-specifiers abstract-declarator$_{\text{opt}}$
identifier-list:
identifier
identifier-list `,` *identifier*

Semantics

2 Each declarator declares one identifier, and asserts that when an operand of the same form as the declarator appears in an expression, it designates a function or object with the scope, storage duration, and type indicated by the declaration specifiers.

3 A *full declarator* is a declarator that is not part of another declarator. If, in the nested sequence of declarators in a full declarator, there is a declarator specifying a variable length array type, the type specified by the full declarator is said to be *variably modified*. Furthermore, any type derived by declarator type derivation from a variably modified type is itself variably modified.

4 In the following subclauses, consider a declaration

`T D1`

where `T` contains the declaration specifiers that specify a type *T* (such as **`int`**) and `D1` is a declarator that contains an identifier *ident*. The type specified for the identifier *ident* in the various forms of declarator is described inductively using this notation.

5 If, in the declaration "`T D1`", `D1` has the form

identifier

then the type specified for *ident* is *T*.

6 If, in the declaration "`T D1`", `D1` has the form

`(` `D` `)`

then *ident* has the type specified by the declaration "`T D`". Thus, a declarator in parentheses is identical to the unparenthesized declarator, but the binding of complicated declarators may be altered by parentheses.

Implementation limits

7 As discussed in 5.2.4.1, an implementation may limit the number of pointer, array, and function declarators that modify an arithmetic, structure, union, or **`void`** type, either directly or via one or more **`typedef`** s.

Forward references: array declarators (6.7.6.2), type definitions (6.7.8)~~.~~, type inference (6.7.11).

6.7.6.1 Pointer declarators

Semantics

1 If, in the declaration "`T D1`", `D1` has the form

`*` *type-qualifier-list*$_{\text{opt}}$ `D`

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the

```
    }
```

Forward references: function declarators (6.7.6.3), function definitions (6.9.1), initialization (6.7.10).

6.7.6.3 Function declarators (including prototypes)

Constraints

1 A function declarator shall not specify a return type that is a function type or an array type.

2 The only storage-class ~~specifier~~ specifiers that shall occur in a parameter declaration ~~is~~ are **`auto`** and **`register`**.

3 An identifier list in a function declarator that is not part of a definition of that function shall be empty. A parameter declaration without type specifier shall not be formed, unless it includes the storage class specifier **`auto`** and unless it appears in the parameter list of a lambda expression.

4 After adjustment, the parameters in a parameter type list in a function declarator that is part of a definition of that function shall not have incomplete type.

Semantics

5 If, in the declaration "`T D1`", `D1` has the form

`D ( `*parameter-type-list*` )`

or

`D ( `*identifier-list*$_{\text{opt}}$` )`

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the type specified for *ident* is "*derived-declarator-type-list* function returning the unqualified version of *T*".

6 A parameter type list specifies the types of, and may declare identifiers for, the parameters of the function.

7 ~~A~~ After the declared types of all parameters have been determined in order of declaration, any declaration of a parameter as "array of *type*" shall be adjusted to "qualified pointer to *type*", where the type qualifiers (if any) are those specified within the `[` and `]` of the array type derivation. If the keyword **`static`** also appears within the `[` and `]` of the array type derivation, then for each call to the function, the value of the corresponding actual argument shall provide access to the first element of an array with at least as many elements as specified by the size expression.

8 A declaration of a parameter as "function returning *type*" shall be adjusted to "pointer to function returning *type*", as in 6.3.2.1.

9 If the list terminates with an ellipsis (`, ...`), no information about the number or types of the parameters after the comma is supplied.[159)]

10 The special case of an unnamed parameter of type **`void`** as the only item in the list specifies that the function has no parameters.

11 If, in a parameter declaration, an identifier can be treated either as a typedef name or as a parameter name, it shall be taken as a typedef name.

12 If the function declarator is not part of a definition of that function, parameters may have incomplete type and may use the `[*]` notation in their sequences of declarator specifiers to specify variable length array types.

13 The storage-class specifier in the declaration specifiers for a parameter declaration, if present, is ignored unless the declared parameter is one of the members of the parameter type list for a function definition.

14 An identifier list declares only the identifiers of the parameters of the function. An empty list in a function declarator that is part of a definition of that function specifies that the function has no parameters. The empty list in a function declarator that is not part of a definition of that function

[159)] The macros defined in the `<stdarg.h>` header (7.16) can be used to access arguments that correspond to the ellipsis.

name respectively the types (a) **int**, (b) pointer to **int**, (c) array of three pointers to **int**, (d) pointer to an array of three **int** s, (e) pointer to a variable length array of an unspecified number of **int** s, (f) function with no parameter specification returning a pointer to **int**, (g) pointer to function with no parameters returning an **int**, and (h) array of an unspecified number of constant pointers to functions, each with one parameter that has type **unsigned int** and an unspecified number of other parameters, returning an **int**.

6.7.8 Type definitions

Syntax

1 *typedef-name:*
identifier

Constraints

2 If a typedef name specifies a variably modified type then it shall have block scope.

Semantics

3 In a declaration whose storage-class specifier is **typedef**, each declarator defines an identifier to be a typedef name that denotes the type specified for the identifier in the way described in 6.7.6. Any array size expressions associated with variable length array declarators are evaluated each time the declaration of the typedef name is reached in the order of execution. A **typedef** declaration does not introduce a new type, only a synonym for the type so specified. That is, in the following declarations:

```
typedef T type_ident;
type_ident D;
```

`type_ident` is defined as a typedef name with the type specified by the declaration specifiers in `T` (known as *T*), and the identifier in `D` has the type "*derived-declarator-type-list T*" where the *derived-declarator-type-list* is specified by the declarators of `D`. A typedef name shares the same name space as other identifiers declared in ordinary declarators. If the identifier is redeclared in an enclosed block the inner declaration shall not be underspecified.

4 **EXAMPLE 1** After

```
typedef int MILES, KLICKSP();
typedef struct { double hi, lo; } range;
```

the constructions

```
MILES distance;
extern KLICKSP *metricp;
range x;
range z, *zp;
```

are all valid declarations. The type of `distance` is **int**, that of `metricp` is "pointer to function with no parameter specification returning **int**", and that of `x` and `z` is the specified structure; `zp` is a pointer to such a structure. The object `distance` has a type compatible with any other **int** object.

5 **EXAMPLE 2** After the declarations

```
typedef struct s1 { int x; } t1, *tp1;
typedef struct s2 { int x; } t2, *tp2;
```

type `t1` and the type pointed to by `tp1` are compatible. Type `t1` is also compatible with type **struct** `s1`, but not compatible with the types **struct** `s2`, `t2`, the type pointed to by `tp2`, or **int**.

6 **EXAMPLE 3** The following obscure constructions

```
typedef signed int t;
typedef int plain;
struct tag {
      unsigned t:4;
```

```
struct S {
      int i;
      struct T t;
};

struct T x = {.l = 43, .k = 42, };

void f(void)
{
      struct S l = { 1, .t = x, .t.l = 41, };
}
```

The value of `l.t.k` is 42, because implicit initialization does not override explicit initialization.

37 **EXAMPLE 13** Space can be "allocated" from both ends of an array by using a single designator:

```
int a[MAX] = {
      1, 3, 5, 7, 9, [MAX-5] = 8, 6, 4, 2, 0
};
```

38 In the above, if `MAX` is greater than ten, there will be some zero-valued elements in the middle; if it is less than ten, some of the values provided by the first five initializers will be overridden by the second five.

39 **EXAMPLE 14** Any member of a union can be initialized:

```
union { /* ... */ } u = {.any_member = 42 };
```

Forward references: common definitions <stddef.h> (7.19).

6.7.11 Type inference

Constraints

1 An underspecified declaration shall contain the storage class specifier **auto**.

2 For an identifier that is declared but not defined by an underspecified declaration, a prior definition shall be be visible. For an underspecified declaration which is not the declaration of a parameter, an init-declarator corresponding to the definition of an object shall have of one of the forms

declarator **=** *assignment-expression*
declarator **= {** *assignment-expression* **}**
declarator **= {** *assignment-expression* **, }**

such that the declarator does not declare an array.[169] If the assignment expression has lambda type that type shall be complete, the declaration shall only declare one object and shall only consist of storage class specifiers, qualifiers, the identifier that is to be declared, and the initializer.

3 Unless it is the definition of an object with an assignment expression of lambda type as above, prior to an underspecified declaration there shall exist a **typeof** specifier *type* that if used to replace the **auto** specifier makes the adjusted declaration a valid declaration.[170] If it is also the definition of a function the return type shall be determined from **return** statements (or the lack thereof) as specified in 6.9.1. Otherwise, *type* shall be such that for all defined objects the assignment expression in the corresponding init-declarator, after possible lvalue, array-to-pointer or function-to-pointer conversion, has the non-atomic, unqualified type of the declared object.

Description

4 Although there is no syntax derivation to form declarators of lambda type, a value λ of lambda type `L` can be used as assignment expression to initialize an underspecified object declaration and as the return value of an underspecified function. The inferred type then is `L`, possibly qualified, and the visibility of `L` extends to the visibility scope of the declared object or function. Otherwise,

[169] The scope rules as described in 6.2.1 also prohibit the use of the identifier of the declarator within the assignment expression.

[170] The qualification of the type of an lvalue that is the assignment expression, or the fact that it is atomic, can never be used to infer such a property of the type of the defined object.

in an underspecified declaration the type of the declared identifiers is the type after the declaration would have been adjusted by a choice for *type* as described in the constraints. If the declaration is also an object definition, each assignment expressions that is used to determine the type and initial value of an object is evaluated exactly once each time the declaration is met.

5 **NOTE 1** Because of the relatively complex syntax and semantics of type specifiers, the requirements for *type* use a **typeof** specifier. For an underspecified declaration

```
auto x = v;
```

in many situations a non-atomic unqualified type *type* as required can be found by using **typeof** with a comma operator as in **typeof**((**void**)0, v) and the adjusted definition as follows would be valid:

```
typeof((void)0, v) x = v;
```

This is for example the case if the identifier or tag name of the type of the initializer expression v in the initializer of x is shadowed. In contrast to that, if v is a bit-field member to which the implementation assigns a type that has no declaration syntax outside a structure or union declaration, no **typeof** specifier exists and the underspecified declaration as above is invalid. The indicated adjustment with **typeof** doesn't imply that v is evaluated twice, even if it has a variably modified type.

6 **NOTE 2** If an underspecified declaration that also defines a structure or union type is valid or not depends on a prior visibility of a type with the same tag.

```
auto p = (struct s { int a; } *)0;
```

Here a replacement of **auto** by **typeof** without duplication of the definition would be as follows.

```
typeof(struct s *) p = (struct s { int a; } *)0;
```

Such a declaration is valid if no prior definition of a structure or union type with tag s is visible, because then **struct** s is a forward declaration of the following definition. If on the other hand a declaration **struct** s in an surrounding scope is visible, the two types are different and the declaration is invalid. A direct use of the structure definition as the type specifier ensures the validity of the declaration.

```
struct s { int a; } * p = 0;
```

7 **NOTE 3** For most assignment expressions of integer or floating point type, there are several choices for *type* that would make such a declaration valid. The second part of the constraint ensures that among these either a unique type is determined such that none of the initializers needs further conversion, or, that the declaration is invalid because no **typeof** expression exists that has exactly the required type.

8 **NOTE 4** For the correspondence of the declared type of an object and the type of its initializer, having compatible types is not sufficient. For example integer types of the same rank and signedness but that are nevertheless different types are not considered. Thus, the validity of an underspecified declaration that declares several identifiers and that uses initializer expressions for which this standard does not specify a unique type for all implementations, such as integer literals or bit-field members, depends on the concrete choice of these types by the implementation. Where possible, it is recommended to split such an underspecified declaration into several to achieve full portability.

9 **EXAMPLE 1** Consider the following file scope definitions:

```
static auto a = 3.5;
auto * p = &a;
```

They are interpreted as if they had been written as:

```
static double a = 3.5;
double * p = &a;
```

So effectively a is a **double** and p is a **double***.

Both identifiers can later be redeclared as long as such a declaration is consistent with the previous ones. For example declarations as the following

```
   extern auto a;
   extern auto p;
```

may be used inside a block where the file scope declarations are shadowed by declarations in an enclosing block.

10 **EXAMPLE 2** Declarations that are the definition of several objects, may make type inferrence difficult and not portable.

```
   enum A { aVal, } aObj = aVal;
   enum B { bVal, } bObj = bVal;
   int  au = aObj, bu = bObj;  // valid, values have type convertible to int
   auto ax = aObj, bx = bObj;  // invalid, same rank but different types
   auto ay = aObj;             // valid, ay has type enum A
   auto by = bObj;             // valid, by has type enum B
   auto az = aVal, bz = bVal;  // valid, az and bz have type int
   struct set { int bits:6; } X = { .bits = 37, };
   auto k = 37, m = X.bits;    // possibly valid or invalid
```

Here, the definitions of `ax` and `bx` cannot be satified with the same **`typeof`** as a replacement for **`auto`**; any fixed choice would require the conversion of at least one of the initializer expressions to the other type. For `k` and `m` the difficulty is that `X.bits` may have a signed or an unsigned type, and that even it is signed it is not specified that the type then is necessarily **`int`**.

11 **EXAMPLE 3** If *type* is a variably modified type, the variable array bounds that are determined by *type* have to be consistent for all objects that are defined by the same underspecified declaration. This consistency can in general not be verified at translation time.

```
   size_t r, s;
   ...
   double aVM[r];
   double bVM[s];
   double cVM[3];
   double dVM[r];
   auto vmPa = &aVM, vmPb = &bVM;    // undefined if r != s
   auto vmPa = &aVM, vmPc = &cVM;    // invalid, even if for some executions r is 3
   auto vmPa = &aVM, vmPd = &dVM;    // valid, same array sizes in all executions
```

12 **EXAMPLE 4** The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
   {
     double a = 7;
     double b = 9;
     {
       double b = b * b;    // undefined, uses uninitialized variable without address
       printf("%g\n", a);   // valid, uses "a" from outer scope, prints 7
       auto a   = a * a;    // invalid, "a" from outer scope is already shadowed
     }
     {
       auto b   = a * a;    // valid, uses "a" from outer scope
       auto a   = b;        // valid, shadows "a" from outer scope
       ...
       printf("%g\n", a);   // valid, uses "a" from inner scope, prints 49
     }
     ...
   }
```

13 **EXAMPLE 5** In the following, `pA` is valid because the type of `A` after array-to-pointer conversion is a pointer type, and `qA` and `rA` are valid because they do not declare arrays but pointers to array.

```
double A[3]   = { 0 };
auto * pA     = A;
auto (*qA)[3] = &A;
auto * rA     = &A;
```

14 **EXAMPLE 6** Type inference can be used to capture the type of a call to a type-generic function and can be used to ensure that the same type as the argument `x` is used.

```
#include <tgmath.h>
auto y = cos(x);
```

If instead the type of `y` is explicitly specified to a different type than `x`, a diagnosis of the mismatch is not enforced.

15 **EXAMPLE 7** A type-generic macro that generalizes the **`div`** functions (7.22.6.2) is defined and used as follows.

```
#define div(X, Y) _Generic((X)+(Y), int: div, long: ldiv, long long: lldiv)((X), (Y))
auto z = div(x, y);
auto q = z.quot;
auto r = z.rem;
```

16 **EXAMPLE 8** Underspecified definitions of objects may occur in all contexts that allow the initializer syntax as described in the constraints. In particular they can be used to ensure type safety of **`for`**-loop controlling expressions.

```
   for (auto i = j; i < 2*j; ++i) {
      ...
   }
```

Here, regardless of the integer rank or signedness of the type of `j`, `i` will have the non-atomic unqualified type of `j`. So, after lvalue conversion and possible promotion, the two operands of the `<` operator in the controlling expression are guaranteed to have the same type, and, in particular, the same signedness.

6.7.12 Static assertions

Syntax

1 *static_assert-declaration:*

`_Static_assert` **`(`** *constant-expression* **`,`** *string-literal* **`)`** **`;`**

Constraints

2 The constant expression shall compare unequal to 0.

Semantics

3 The constant expression shall be an integer constant expression. If the value of the constant expression compares unequal to 0, the declaration has no effect. Otherwise, the constraint is violated and the implementation shall produce a diagnostic message that includes the text of the string literal, except that characters not in the basic source character set are not required to appear in the message.

Forward references: diagnostics (7.2).

6.8 Statements and blocks

Syntax

1 *statement:*

labeled-statement
compound-statement
expression-statement
selection-statement
iteration-statement
jump-statement

Semantics

2 A *statement* specifies an action to be performed. Except as indicated, statements are executed in sequence.

3 A *block* allows a set of declarations and statements to be grouped into one syntactic unit. The initializers of objects that have automatic storage duration, and the variable length array declarators of ordinary identifiers with block scope, are evaluated and the values are stored in the objects (including storing an indeterminate value in objects without an initializer) each time the declaration is reached in the order of execution, as if it were a statement, and within each declaration in the order that declarators appear.

4 A *full expression* is an expression that is not part of another expression, nor part of a declarator or abstract declarator. There is also an implicit full expression in which the non-constant size expressions for a variably modified type are evaluated; within that full expression, the evaluation of different size expressions are unsequenced with respect to one another. There is a sequence point between the evaluation of a full expression and the evaluation of the next full expression to be evaluated.

5 **NOTE** Each of the following is a full expression:

- a full declarator for a variably modified type,
- an initializer that is not part of a compound literal,
- the expression in an expression statement,
- the controlling expression of a selection statement (**`if`** or **`switch`**),
- the controlling expression of a **`while`** or **`do`** statement,
- each of the (optional) expressions of a **`for`** statement,
- the (optional) expression in a **`return`** statement.

While a constant expression satisfies the definition of a full expression, evaluating it does not depend on nor produce any side effects, so the sequencing implications of being a full expression are not relevant to a constant expression.

Forward references: expression and null statements (6.8.3), selection statements (6.8.4), iteration statements (6.8.5), the **`return`** statement (6.8.6.4).

6.8.1 Labeled statements

Syntax

1 *labeled-statement:*

identifier **`:`** *statement*
`case` *constant-expression* **`:`** *statement*
`default` **`:`** *statement*

Constraints

2 A **`case`** or **`default`** label shall appear only in a **`switch`** statement ~~.~~ that is associated with the same function body as the statement to which the label is attached.[171)] Further constraints on such labels are discussed under the **`switch`** statement.

[171)] Thus, a label that appears within a lambda expression may only be associated to a switch statement within the body of the lambda.

6.8.5.3 The `for` statement

1 The statement

```
for (clause-1; expression-2; expression-3) statement
```

behaves as follows: The expression *expression-2* is the controlling expression that is evaluated before each execution of the loop body. The expression *expression-3* is evaluated as a void expression after each execution of the loop body. If *clause-1* is a declaration, the scope of any identifiers it declares is the remainder of the declaration and the entire loop, including the other two expressions; it is reached in the order of execution before the first evaluation of the controlling expression. If *clause-1* is an expression, it is evaluated as a void expression before the first evaluation of the controlling expression.[177)]

2 Both *clause-1* and *expression-3* can be omitted. An omitted *expression-2* is replaced by a nonzero constant.

6.8.6 Jump statements

Syntax

1 *jump-statement:*

goto *identifier* **;**
continue ;
break ;
return *expression*$_{\text{opt}}$ **;**

Constraints

2 No jump statement other than **return** shall have a target that is found in another function body.[178)]

Semantics

3 A jump statement causes an unconditional jump to another place.

6.8.6.1 The `goto` statement

Constraints

1 The identifier in a **goto** statement shall name a label located somewhere in the enclosing function body. A **goto** statement shall not jump from outside the scope of an identifier having a variably modified type to inside the scope of that identifier.[179)]

Semantics

2 A **goto** statement causes an unconditional jump to the statement prefixed by the named label in the enclosing function.

3 **EXAMPLE 1** It is sometimes convenient to jump into the middle of a complicated set of statements. The following outline presents one possible approach to a problem based on these three assumptions:

1. The general initialization code accesses objects only visible to the current function.
2. The general initialization code is too large to warrant duplication.
3. The code to determine the next operation is at the head of the loop. (To allow it to be reached by **continue** statements, for example.)

```
/* ... */
goto first_time;
for (;;) {
```

[177)] Thus, *clause-1* specifies initialization for the loop, possibly declaring one or more variables for use in the loop; the controlling expression, *expression-2*, specifies an evaluation made before each iteration, such that execution of the loop continues until the expression compares equal to 0; and *expression-3* specifies an operation (such as incrementing) that is performed after each iteration.

[178)] Thus jump statements other than **return** may not jump between different functions or cross the boundaries of a lambda expression, that is, they may not jump into or out of the function body of a lambda. Other features such as signals (7.14) and long jumps (7.13) may delegate control to points of the program that do not fall under these constraints.

[179)] The visibility of labels is restricted such that a **goto** statement that jumps into or out of a different function body, even if it is nested within a lambda, is a constraint violation.

```
        // determine next operation
        /* ... */
        if (need to reinitialize) {
              // reinitialize-only code
              /* ... */
        first_time:
              // general initialization code
              /* ... */
              continue;
        }
        // handle other operations
        /* ... */
  }
```

4 **EXAMPLE 2** A **goto** statement is not allowed to jump past any declarations of objects with variably modified types. A jump within the scope, however, is permitted.

```
  goto lab3;              // invalid:  going INTO scope of VLA.
  {
        double a[n];
        a[j] = 4.4;
  lab3:
        a[j] = 3.3;
        goto lab4;        // valid:  going WITHIN scope of VLA.
        a[j] = 5.5;
  lab4:
        a[j] = 6.6;
  }
  goto lab4;              // invalid:  going INTO scope of VLA.
```

6.8.6.2 The **continue** statement

Constraints

1 A **continue** statement shall appear only in or as a loop body ~~.~~ that is associated to the same function body.[180)]

Semantics

2 A **continue** statement causes a jump to the loop-continuation portion of the smallest enclosing iteration statement; that is, to the end of the loop body. More precisely, in each of the statements

```
while (/* ...  */) {
   /* ...  */
   continue;
   /* ...  */
contin:;
}
```

```
do {
   /* ...  */
   continue;
   /* ...  */
contin:;
} while (/* ...  */);
```

```
for (/* ...  */) {
   /* ...  */
   continue;
   /* ...  */
contin:;
}
```

unless the **continue** statement shown is in an enclosed iteration statement (in which case it is interpreted within that statement), it is equivalent to **goto** `contin`;.[181)]

6.8.6.3 The **break** statement

Constraints

1 A **break** statement shall appear only in or as a switch body or loop body ~~.~~ that is associated to the same function body.[182)]

[180)] Thus a **continue** statement by itself may not be used to terminate the execution of the body of a lambda expresssion.
[181)] Following the `contin`: label is a null statement.
[182)] Thus a **break** statement by itself may not be used terminate the execution of the body of a lambda expresssion.

Semantics

2 A **break** statement terminates execution of the smallest enclosing **switch** or iteration statement.

6.8.6.4 The `return` statement

Constraints

1 A **return** statement with an expression shall not appear in a function body whose return type is **void**. A **return** statement without an expression shall only appear in a function body whose return type is **void**.

2 For a function body that corresponds to an underspecified definition of a function or to a lambda, all **return** statements shall provide expressions with a consistent type or none at all. That is, if any **return** statement has an expression, all **return** statements shall have an expression (after lvalue, array-to-pointer or function-to-pointer conversion) with the same type; otherwise all **return** expressions shall have no expression.

Semantics

3 A **return** statement ~~terminates execution of the current function~~ is associated to the innermost function body in which appears. It evaluates the expression, if any, terminates the execution of that function body and returns control to its caller~~. A function~~; if it has an expression, the value of the expression is returned to the caller as the value of the function call expression. A function body may have any number of **return** statements.

4 If a **return** statement with an expression is executed, the value of the expression is returned to the caller as the value of the function call expression. If the expression has a type different from the return type of the function in which it appears, the value is converted as if by assignment to an object having the return type of the function.[183)]

5 For a lambda or for a function that has an underspecified definition, the return type is determined by the lexically first **return** statement, if any, that is associated to the function body and is specified as the type of that expression, if any, after lvalue, array-to-pointer, function-to-pointer conversion, or as **void** if there is no expression.

6 **EXAMPLE** In:

```
struct s { double i; } f(void);
union {
      struct {
            int f1;
            struct s f2;
      } u1;
      struct {
            struct s f3;
            int f4;
      } u2;
} g;

struct s f(void)
{
      return g.u1.f2;
}

/* ... */
g.u2.f3 = f();
```

there is no undefined behavior, although there would be if the assignment were done directly (without using a function call to fetch the value).

[183)] The **return** statement is not an assignment. The overlap restriction of 6.5.16.1 does not apply to the case of function return. The representation of floating-point values can have wider range or precision than implied by the type; a cast can be used to remove this extra range and precision.

6.9 External definitions

Syntax

1 *translation-unit:*
external-declaration
translation-unit external-declaration

external-declaration:
function-definition
declaration

Constraints

2 The storage-class ~~specifiers **auto** and~~ specifier **register** shall not appear in the declaration specifiers in an external declaration.

3 There shall be no more than one external definition for each identifier declared with internal linkage in a translation unit. Moreover, if an identifier declared with internal linkage is used in an expression (other than as a part of the operand of a **sizeof** or **_Alignof** operator whose result is an integer constant), there shall be exactly one external definition for the identifier in the translation unit.

Semantics

4 As discussed in 5.1.1.1, the unit of program text after preprocessing is a translation unit, which consists of a sequence of external declarations. These are described as "external" because they appear outside any function (and hence have file scope). As discussed in 6.7, a declaration that also causes storage to be reserved for an object or a function named by the identifier is a definition.

5 An *external definition* is an external declaration that is also a definition of a function (other than an inline definition) or an object. If an identifier declared with external linkage is used in an expression (other than as part of the operand of a **sizeof** or **_Alignof** operator whose result is an integer constant), somewhere in the entire program there shall be exactly one external definition for the identifier; otherwise, there shall be no more than one.[184)]

6.9.1 Function definitions

Syntax

1 *function-definition:*
declaration-specifiers declarator declaration-list$_{\text{opt}}$ *compound-statement*

declaration-list:
declaration
declaration-list declaration

Constraints

2 The identifier declared in a function definition (which is the name of the function) shall have a function type, as specified by the declarator portion of the function definition.[185)]

3 The return type of a function shall be **void** or a complete object type other than array type.

4 The storage-class specifier, if any, in the declaration specifiers shall be either **extern** or **static**, possibly combined with **auto**.

5 If the declarator includes a parameter type list, the declaration of each parameter shall include an identifier, except for the special case of a parameter list consisting of a single parameter of type **void**, in which case there shall not be an identifier. No declaration list shall follow.

[184)] Thus, if an identifier declared with external linkage is not used in an expression, there need be no external definition for it.

6 If the declarator includes an identifier list, each declaration in the declaration list shall have at least one declarator, those declarators shall declare only identifiers from the identifier list, and every identifier in the identifier list shall be declared. An identifier declared as a typedef name shall not be redeclared as a parameter. The declarations in the declaration list shall contain no storage-class specifier other than **`register`** and no initializations.

7 An underspecified function definition shall contain an **`auto`** storage class specifier. The return type for such a function is determined as described for the **`return`** statement (6.8.6.4) and shall be visible prior to the function definition.

Semantics

8 If **`auto`** appears as a storage-class specifier it is ignored for the purpose of determining a storage class or linkage of the function. It then only indicates that the return type of the function may be inferred from **`return`** statements or the lack thereof, see 6.8.6.4.

9 The declarator in a function definition specifies the name of the function being defined and the identifiers of its parameters. If the declarator includes a parameter type list, the list also specifies the types of all the parameters; such a declarator (possibly adjusted by an inferred type specifier) also serves as a function prototype for later calls to the same function in the same translation unit. If the declarator includes an identifier list,[186] the types of the parameters shall be declared in a following declaration list. In either case, the type of each parameter is adjusted as described in 6.7.6.3 for a parameter type list; the resulting type shall be a complete object type.

10 If a function that accepts a variable number of arguments is defined without a parameter type list that ends with the ellipsis notation, the behavior is undefined.

11 Each parameter has automatic storage duration; its identifier is an lvalue.[187] The layout of the storage for parameters is unspecified.

12 On entry to the function, the size expressions of each variably modified parameter are evaluated and the value of each argument expression is converted to the type of the corresponding parameter as if by assignment. (Array expressions and function designators as arguments were converted to pointers before the call.)

13 After all parameters have been assigned, the compound statement that constitutes the body of the function definition is executed.

14 Unless otherwise specified, if the } that terminates a function is reached, and the value of the function call is used by the caller, the behavior is undefined.

15 Provided the constraints above are respected, the return type of an underspecified function definition is adjusted as if the corresponding type specifier had been inserted in the definition. The type of such a function is incomplete within the function body until the lexically first **`return`** statement that it contains, if any, or until the end of the function body, otherwise.[188]

16 **NOTE** In a function definition, the type of the function and its prototype cannot be inherited from a typedef:

```
typedef int F(void);                // type F is "function with no parameters
                                    // returning int"
F f, g;                             // f and g both have type compatible with F
F f { /* ... */ }                   // WRONG: syntax/constraint error
F g() { /* ... */ }                 // WRONG: declares that g returns a function
int f(void) { /* ... */ }           // RIGHT: f has type compatible with F
int g() { /* ... */ }               // RIGHT: g has type compatible with F
F *e(void) { /* ... */ }            // e returns a pointer to a function
F *((e))(void) { /* ... */ }        // same:  parentheses irrelevant
int (*fp)(void);                    // fp points to a function that has type F
F *Fp;                              // Fp points to a function that has type F
```

185) ~~The intent is that the type category in a function definition cannot be inherited from a typedef.~~

186) See "future language directions" (6.11.7).

187) A parameter identifier cannot be redeclared in the function body except in an enclosed block.

188) This means that such a function cannot be used for direct recursion before or within the first return statement.

17 **EXAMPLE 1** In the following:

```
extern int max(int a, int b)
{
      return a  >  b ? a: b;
}
```

extern is the storage-class specifier and **int** is the type specifier; `max(int a, int b)` is the function declarator; and

```
{ return a  >  b ? a: b; }
```

is the function body. The following similar definition uses the identifier-list form for the parameter declarations:

```
extern int max(a, b)
int a, b;
{
      return a  >  b ? a: b;
}
```

Here **int** `a, b;` is the declaration list for the parameters. The difference between these two definitions is that the first form acts as a prototype declaration that forces conversion of the arguments of subsequent calls to the function, whereas the second form does not.

18 **EXAMPLE 2** To pass one function to another, one might say

```
int f(void);
/* ... */
g(f);
```

Then the definition of `g` might read

```
void g(int (*funcp)(void))
{
      /* ... */
      (*funcp)(); /* or funcp(); ...*/
}
```

or, equivalently,

```
void g(int func(void))
{
      /* ... */
      func(); /* or (*func)(); ...*/
}
```

19 **EXAMPLE 3** Consider the following function that computes the maximum value of two parameters that have integer types T and S.

```
   inline auto max(T, S); // invalid: no definition visible
   ...
   inline auto max(T a, S b){
     return (a < 0)
            ? ((b <  0) ? ((a < b) ? b : a) : b)
            : ((b >= 0) ? ((a < b) ? b : a) : a);
   }
   ...
   // valid: definition visible
   extern auto max(T, S); // forces definition to be external
   auto max(T, S);        // same
   auto max();            // same
```

The **return** expression performs default arithmetic conversion to determine a type that can hold the maximum value and is at least as wide as **int**. The function definition is adjusted to that return type. This property holds regardless if types T and S have the same or different signedness.

The first forward declaration of the function is invalid, because an **auto** type function declaration that is not a definition is only valid if the definition of the function is visible. In contrast to that, the **extern** declaration and the two following equivalent ones are valid because they follow the definition and thus the inferred return type is known. Thereby in is ensured that the translation unit provides an external definition of the function.

20 **EXAMPLE 4** The following function computes the sum over an array of integers of type T and returns the value as the promoted type of T.

```
inline
auto sum(size_t n, T A[n]){
  switch(n) {
    case 0:
      return +((T)0);                          // return the promoted type
    case 1:
      return +A[0];                            // return the promoted type
    default:
      return sum(n/2, A) + sum(n - n/2, &A[n/2]);  // valid recursion
  }
}
```

If instead sum would have bee defined with a prototype as follows

```
T sum(size_t n, T A[n]);
```

for a narrow type T such as **unsigned char**, the return type and result would be different from the previous. In particular, the result of the addition would have been converted back from the promoted type to T before each **return**, possibly leading to a surprising overall results. Also, specifying the promoted type of a narrow type T explicitly can be tedious because that type depends on properties of the execution platform.

6.9.2 External object definitions

Semantics

1 If the declaration of an identifier for an object has file scope and an initializer, the declaration is an external definition for the identifier.

2 A declaration of an identifier for an object that has file scope without an initializer, and without a storage-class specifier or with the storage-class specifier **static**, constitutes a *tentative definition*. If a translation unit contains one or more tentative definitions for an identifier, and the translation unit contains no external definition for that identifier, then the behavior is exactly as if the translation unit contains a file scope declaration of that identifier, with the composite type as of the end of the translation unit, with an initializer equal to 0.

3 If the declaration of an identifier for an object is a tentative definition and has internal linkage, the declared type shall not be an incomplete type.

4 **EXAMPLE 1**

```
int i1 = 1;         // definition, external linkage
static int i2 = 2;  // definition, internal linkage
extern int i3 = 3;  // definition, external linkage
int i4;             // tentative definition, external linkage
static int i5;      // tentative definition, internal linkage

int i1;             // valid tentative definition, refers to previous
int i2;             // 6.2.2 renders undefined, linkage disagreement
int i3;             // valid tentative definition, refers to previous
int i4;             // valid tentative definition, refers to previous
int i5;             // 6.2.2 renders undefined, linkage disagreement

extern int i1;      // refers to previous, whose linkage is external
```

— If an argument to a function has an invalid value (such as a value outside the domain of the function, or a pointer outside the address space of the program, or a null pointer, or a pointer to non-modifiable storage when the corresponding parameter is not const-qualified) or a type (after default argument promotion) not expected by a function with a variable number of arguments, the behavior is undefined.

— If a function argument is described as being an array, the pointer actually passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are in fact valid.

— Any function declared in a header may be additionally implemented as a function-like macro defined in the header, so if a library function is declared explicitly when its header is included, one of the techniques shown below can be used to ensure the declaration is not affected by such a macro. Any macro definition of a function can be suppressed locally by enclosing the name of the function in parentheses, because the name is then not followed by the left parenthesis that indicates expansion of a macro function name. For the same syntactic reason, it is permitted to take the address of a library function even if it is also defined as a macro.[211] The use of **`#undef`** to remove any macro definition will also ensure that an actual function is referred to.

— Any invocation of a library function that is implemented as a macro shall expand to code that evaluates each of its arguments exactly once, fully protected by parentheses where necessary, so it is generally safe to use arbitrary expressions as arguments.[212]

— Likewise, those function-like macros described in the following subclauses may be invoked in an expression anywhere a function with a compatible return type could be called.[213]

— All object-like macros listed as expanding to integer constant expressions shall additionally be suitable for use in **`#if`** preprocessing directives.

2 Provided that a library function can be declared without reference to any type defined in a header, it is also permissible to declare the function and use it without including its associated header.

3 There is a sequence point immediately before a library function returns.

4 The functions in the standard library are not guaranteed to be reentrant and may modify objects with static or thread storage duration.[214]

5 Unless explicitly stated otherwise in the detailed descriptions that follow, library functions shall prevent data races as follows: A library function shall not directly or indirectly access objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's arguments. A library function shall not directly or indirectly modify objects accessible by threads other than the current thread unless the objects are accessed directly

[211] This means that an implementation is required to provide an actual function for each library function, even if it also provides a macro for that function.

[212] Such macros might not contain the sequence points that the corresponding function calls do. Nevertheless, it is recommended that implementations provide the same sequencing properties as for a function call, by, for example, wrapping the macro expansion in a suitable lambda expression.

[213] Because external identifiers and some macro names beginning with an underscore are reserved, implementations can provide special semantics for such names. For example, the identifier `_BUILTIN_abs` could be used to indicate generation of in-line code for the **`abs`** function. Thus, the appropriate header could specify

```
#define abs(x) _BUILTIN_abs(x)
```

for a compiler whose code generator will accept it.

In this manner, a user desiring to guarantee that a given library function such as **`abs`** will be a genuine function can write

```
#undef abs
```

whether the implementation's header provides a macro implementation of **`abs`** or a built-in implementation. The prototype for the function, which precedes and is hidden by any macro definition, is thereby revealed also.

[214] Thus, a signal handler cannot, in general, call standard library functions.

Description

2 The **longjmp** function restores the environment saved by the most recent invocation of the **setjmp** macro in the same invocation of the program with the corresponding **jmp_buf** argument. If there has been no such invocation, or if the invocation was from another thread of execution, or if the function body containing the invocation of the **setjmp** macro has terminated execution[274] in the interim, or if the invocation of the **setjmp** macro was within the scope of an identifier with variably modified type and execution has left that scope in the interim, the behavior is undefined.

3 All accessible objects have values, and all other components of the abstract machine[275] have state, as of the time the **longjmp** function was called, except that the values of objects of automatic storage duration that are local to the function containing the invocation of the corresponding **setjmp** macro[276] that do not have volatile-qualified type and have been changed between the **setjmp** invocation and **longjmp** call are indeterminate.

Returns

4 After **longjmp** is completed, thread execution continues as if the corresponding invocation of the **setjmp** macro had just returned the value specified by `val`. The **longjmp** function cannot cause the **setjmp** macro to return the value 0; if `val` is 0, the **setjmp** macro returns the value 1.

5 **EXAMPLE** The **longjmp** function that returns control back to the point of the **setjmp** invocation might cause memory associated with a variable length array object to be squandered.

```
#include <setjmp.h>
jmp_buf buf;
void g(int n);
void h(int n);
int n = 6;

void f(void)
{
      int x[n];          // valid:  f is not terminated
      setjmp(buf);
      g(n);
}

void g(int n)
{
      int a[n];          // a may remain allocated
      h(n);
}

void h(int n)
{
      int b[n];          // b may remain allocated
      longjmp(buf, 2);   // might cause memory loss
}
```

[274] For example, by executing a **return** statement or because another **longjmp** call has caused a transfer to a **setjmp** invocation in a function or lambda earlier in the set of nested calls.

[275] This includes, but is not limited to, the floating-point status flags and the state of open files.

[276] Such a function contains the call to **setjmp** either directly or within a set of nested lambdas. All local variables of the function and the nested lambdas that have been modified between the corresponding calls to **setjmp** and **longjmp** function are affected.

7.14 Signal handling <signal.h>

1 The header <signal.h> declares a type and two functions and defines several macros, for handling various *signals* (conditions that may be reported during program execution).

2 The type defined is

```
sig_atomic_t
```

which is the (possibly volatile-qualified) integer type of an object that can be accessed as an atomic entity, even in the presence of asynchronous interrupts.

3 The macros defined are

```
SIG_DFL
SIG_ERR
SIG_IGN
```

which expand to constant expressions with distinct values that have type compatible with the second argument to, and the return value of, the **signal** function, and whose values compare unequal to the address of any declarable function; and the following, which expand to positive integer constant expressions with type **int** and distinct values that are the signal numbers, each corresponding to the specified condition:

SIGABRT abnormal termination, such as is initiated by the **abort** function

SIGFPE an erroneous arithmetic operation, such as zero divide or an operation resulting in overflow

SIGILL detection of an invalid function image, such as an invalid instruction

SIGINT receipt of an interactive attention signal

SIGSEGV an invalid access to storage

SIGTERM a termination request sent to the program

4 An implementation need not generate any of these signals, except as a result of explicit calls to the **raise** function. Additional signals and pointers to undeclarable functions, with macro definitions beginning, respectively, with the letters **SIG** and an uppercase letter or with **SIG_** and an uppercase letter,[277] may also be specified by the implementation. The complete set of signals, their semantics, and their default handling is implementation-defined; all signal numbers shall be positive.

7.14.1 Specify signal handling

7.14.1.1 The signal function

Synopsis

1

```
#include <signal.h>
void (*signal(int sig, void (*func)(int)))(int);
```

Description

2 The **signal** function chooses one of three ways in which receipt of the signal number sig is to be subsequently handled. If the value of func is **SIG_DFL**, default handling for that signal will occur. If the value of func is **SIG_IGN**, the signal will be ignored. Otherwise, func shall point to a function or shall be the result of a conversion of a function literal to a function pointer. The function or lambda value is then to be called when that signal occurs~~.~~, An invocation of such a function or function literal because of a signal, or (recursively) of any further functions or lambdas called by that invocation (other than functions in the standard library),[278] is called a *signal handler*.

[277] See "future library directions" (7.31.7). The names of the signal numbers reflect the following terms (respectively): abort, floating-point exception, illegal instruction, interrupt, segmentation violation, and termination.

[278] This includes functions called indirectly via standard library functions (e.g., a **SIGABRT** handler called via the **abort** function).

3 When a signal occurs and `func` points to a function,[279] it is implementation-defined whether the equivalent of `signal(sig, SIG_DFL);` is executed or the implementation prevents some implementation-defined set of signals (at least including `sig`) from occurring until the current signal handling has completed; in the case of **SIGILL**, the implementation may alternatively define that no action is taken. Then the equivalent of `(*func)(sig);` is executed. If and when the function returns, if the value of `sig` is **SIGFPE**, **SIGILL**, **SIGSEGV**, or any other implementation-defined value corresponding to a computational exception, the behavior is undefined; otherwise the program will resume execution at the point it was interrupted.

4 If the signal occurs as the result of calling the **abort** or **raise** function, the signal handler shall not call the **raise** function.

5 If the signal occurs other than as the result of calling the **abort** or **raise** function, the behavior is undefined if the signal handler refers to any object with static or thread storage duration that is not a lock-free atomic object other than by assigning a value to an object declared as **volatile sig_atomic_t**, or the signal handler calls any function in the standard library other than

— the **abort** function,

— the **_Exit** function,

— the **quick_exit** function,

— the functions in `<stdatomic.h>` (except where explicitly stated otherwise) when the atomic arguments are lock-free,

— the **atomic_is_lock_free** function with any atomic argument, or

— the **signal** function with the first argument equal to the signal number corresponding to the signal that caused the invocation of the handler. Furthermore, if such a call to the **signal** function results in a **SIG_ERR** return, the value of **errno** is indeterminate.[280]

6 At program startup, the equivalent of

```
signal(sig, SIG_IGN);
```

may be executed for some signals selected in an implementation-defined manner; the equivalent of

```
signal(sig, SIG_DFL);
```

is executed for all other signals defined by the implementation.

7 Use of this function in a multi-threaded program results in undefined behavior. The implementation shall behave as if no library function calls the **signal** function.

Returns

8 If the request can be honored, the **signal** function returns the value of `func` for the most recent successful call to **signal** for the specified signal `sig`. Otherwise, a value of **SIG_ERR** is returned and a positive value is stored in **errno**.

Forward references: the **abort** function (7.22.4.1), the **exit** function (7.22.4.4), the **_Exit** function (7.22.4.5), the **quick_exit** function (7.22.4.7).

7.14.2 Send signal

7.14.2.1 The raise function

Synopsis

1
```
#include <signal.h>
int raise(int sig);
```

[279] Or, equivalently, it is the result of a conversion of a function literal to a function pointer.
[280] If any signal is generated by an asynchronous signal handler, the behavior is undefined.

7.16 Variable arguments <stdarg.h>

1 The header <stdarg.h> declares a type and defines four macros, for advancing through a list of arguments whose number and types are not known to the called function when it is translated.

2 A function may be called with a variable number of arguments of varying types. As described in 6.9.1, its parameter list contains one or more parameters. The rightmost parameter plays a special role in the access mechanism, and will be designated *parmN* in this description.

3 The type declared is

```
va_list
```

which is a complete object type suitable for holding information needed by the macros **va_start**, **va_arg**, **va_end**, and **va_copy**. If access to the varying arguments is desired, the called function shall declare an object (generally referred to as ap in this subclause) having type **va_list**. The object ap may be passed as an argument to another function ~~; if that function~~ call; if the called function or lambda invokes the **va_arg** macro with parameter ap, the value of ap in the calling function or lambda is indeterminate and shall be passed to the **va_end** macro prior to any further reference to ap.[281)]

4 **NOTE** Because the ... parameter syntax is not valid for lambda expressions, these macros can never be applied directly to process a variable list of arguments to the call of a lambda. In contrast to that, the type **va_list** itself can be a parameter type of a lambda expression to process the argument list of a function.

7.16.1 Variable argument list access macros

1 The **va_start** and **va_arg** macros described in this subclause shall be implemented as macros, not functions. It is unspecified whether **va_copy** and **va_end** are macros or identifiers declared with external linkage. If a macro definition is suppressed in order to access an actual function, or a program defines an external identifier with the same name, the behavior is undefined. Each invocation of the **va_start** and **va_copy** macros shall be matched by a corresponding invocation of the **va_end** macro in the same function or lambda expression.

7.16.1.1 The va_arg macro

Synopsis

1
```
#include <stdarg.h>
type va_arg(va_list ap, type);
```

Description

2 The **va_arg** macro expands to an expression that has the specified type and the value of the next argument in the call. The parameter ap shall have been initialized by the **va_start** or **va_copy** macro (without an intervening invocation of the **va_end** macro for the same ap). Each invocation of the **va_arg** macro modifies ap so that the values of successive arguments are returned in turn. The parameter *type* shall be a type name specified such that the type of a pointer to an object that has the specified type can be obtained simply by postfixing a * to *type*. If there is no actual next argument, or if *type* is not compatible with the type of the actual next argument (as promoted according to the default argument promotions), the behavior is undefined, except for the following cases:

— one type is a signed integer type, the other type is the corresponding unsigned integer type, and the value is representable in both types;

— one type is pointer to **void** and the other is a pointer to a character type.

Returns

3 The first invocation of the **va_arg** macro after that of the **va_start** macro returns the value of the argument after that specified by *parmN*. Successive invocations return the values of the remaining arguments in succession.

[281)] It is permitted to create a pointer to a **va_list** and pass that pointer to another function or lambda, in which case the ~~original~~ calling function or lambda can make further use of the original list after the other function returns.

7.16.1.2 The `va_copy` macro

Synopsis

1
```
#include <stdarg.h>
void va_copy(va_list dest, va_list src);
```

Description

2 The **`va_copy`** macro initializes `dest` as a copy of `src`, as if the **`va_start`** macro had been applied to `dest` followed by the same sequence of uses of the **`va_arg`** macro as had previously been used to reach the present state of `src`. Neither the **`va_copy`** nor **`va_start`** macro shall be invoked to reinitialize `dest` without an intervening invocation of the **`va_end`** macro for the same `dest`.

Returns

3 The **`va_copy`** macro returns no value.

7.16.1.3 The `va_end` macro

Synopsis

1
```
#include <stdarg.h>
void va_end(va_list ap);
```

Description

2 The **`va_end`** macro facilitates a normal return from the function whose variable argument list was referred to by the expansion of the **`va_start`** macro, or the function or lambda expression containing the expansion of the **`va_copy`** macro, that initialized the **`va_list`** `ap`. The **`va_end`** macro may modify `ap` so that it is no longer usable (without being reinitialized by the **`va_start`** or **`va_copy`** macro). If there is no corresponding invocation of the **`va_start`** or **`va_copy`** macro, or if the **`va_end`** macro is not invoked before the return, the behavior is undefined.

Returns

3 The **`va_end`** macro returns no value.

7.16.1.4 The `va_start` macro

Synopsis

1
```
#include <stdarg.h>
void va_start(va_list ap, parmN);
```

Description

2 The **`va_start`** macro shall be invoked before any access to the unnamed arguments.

3 The **`va_start`** macro initializes `ap` for subsequent use by the **`va_arg`** and **`va_end`** macros. Neither the **`va_start`** nor **`va_copy`** macro shall be invoked to reinitialize `ap` without an intervening invocation of the **`va_end`** macro for the same `ap`.

4 The parameter *parmN* is the identifier of the rightmost parameter in the variable parameter list in the function definition (the one just before the `, ...`). If the parameter *parmN* is declared with the **`register`** storage class, with a function or array type, or with a type that is not compatible with the type that results after application of the default argument promotions, the behavior is undefined.

Returns

5 The **`va_start`** macro returns no value.

6 **EXAMPLE 1** The function `f1` gathers into an array a list of arguments that are pointers to strings (but not more than `MAXARGS` arguments), then passes the array as a single argument to function `f2`. The number of pointers is specified by the first argument to `f1`.

Returns

4 The **realloc** function returns a pointer to the new object (which may have the same value as a pointer to the old object), or a null pointer if the new object has not been allocated.

7.22.4 Communication with the environment

7.22.4.1 The abort function

Synopsis

1
```
#include <stdlib.h>
_Noreturn void abort(void);
```

Description

2 The **abort** function causes abnormal program termination to occur, unless the signal **SIGABRT** is being caught and the signal handler does not return. Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined. An implementation-defined form of the status *unsuccessful termination* is returned to the host environment by means of the function call **raise**(**SIGABRT**).

Returns

3 The **abort** function does not return to its caller.

7.22.4.2 The atexit function

Synopsis

1
```
#include <stdlib.h>
int atexit(void (*func)(void));
```

Description

2 The **atexit** function registers the function or function literal pointed to by func, to be called without arguments at normal program termination.[326)] It is unspecified whether a call to the **atexit** function that does not happen before the **exit** function is called will succeed.

Environmental limits

3 The implementation shall support the registration of at least 32 ~~functions~~function pointers.

Returns

4 The **atexit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **at_quick_exit** function (7.22.4.3), the **exit** function (7.22.4.4).

7.22.4.3 The at_quick_exit function

Synopsis

1
```
#include <stdlib.h>
int at_quick_exit(void (*func)(void));
```

Description

2 The **at_quick_exit** function registers the function or function literal pointed to by func, to be called without arguments should **quick_exit** be called.[327)] It is unspecified whether a call to the **at_quick_exit** function that does not happen before the **quick_exit** function is called will succeed.

[326)] The **atexit** function registrations are distinct from the **at_quick_exit** registrations, so applications might need to call both registration functions with the same argument.

[327)] The **at_quick_exit** function registrations are distinct from the **atexit** registrations, so applications might need to call both registration functions with the same argument.

Environmental limits

3 The implementation shall support the registration of at least 32 ~~functions~~function pointers.

Returns

4 The **at_quick_exit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **quick_exit** function (7.22.4.7).

7.22.4.4 The exit function

Synopsis

1
```
#include <stdlib.h>
_Noreturn void exit(int status);
```

Description

2 The **exit** function causes normal program termination to occur. No ~~functions~~ function pointers registered by the **at_quick_exit** function are called. If a program calls the **exit** function more than once, or calls the **quick_exit** function in addition to the **exit** function, the behavior is undefined.

3 First, all ~~functions~~ function pointers registered by the **atexit** function are called, in the reverse order of their registration,[328)] except that a function pointer is called after any previously registered ~~functions~~ function pointers that had already been called at the time it was registered. If, during the call to any such function or function literal, a call to the **longjmp** function is made that would terminate the call to the registered function or function literal, the behavior is undefined.

4 Next, all open streams with unwritten buffered data are flushed, all open streams are closed, and all files created by the **tmpfile** function are removed.

5 Finally, control is returned to the host environment. If the value of `status` is zero or **EXIT_SUCCESS**, an implementation-defined form of the status *successful termination* is returned. If the value of `status` is **EXIT_FAILURE**, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.

Returns

6 The **exit** function cannot return to its caller.

7.22.4.5 The _Exit function

Synopsis

1
```
#include <stdlib.h>
_Noreturn void _Exit(int status);
```

Description

2 The **_Exit** function causes normal program termination to occur and control to be returned to the host environment. No ~~functions~~ function pointers registered by the **atexit** function, the **at_quick_exit** function, or signal handlers registered by the **signal** function are called. The status returned to the host environment is determined in the same way as for the **exit** function (7.22.4.4). Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined.

Returns

3 The **_Exit** function cannot return to its caller.

7.22.4.6 The getenv function

Synopsis

1
```
#include <stdlib.h>
```

[328)] Each function is called as many times as it was registered, and in the correct order with respect to other registered ~~functions~~function pointers.

```
char *getenv(const char *name);
```

Description

2 The **getenv** function searches an *environment list*, provided by the host environment, for a string that matches the string pointed to by name. The set of environment names and the method for altering the environment list are implementation-defined. The **getenv** function need not avoid data races with other threads of execution that modify the environment list.[329)]

3 The implementation shall behave as if no library function calls the **getenv** function.

Returns

4 The **getenv** function returns a pointer to a string associated with the matched list member. The string pointed to shall not be modified by the program, but may be overwritten by a subsequent call to the **getenv** function. If the specified name cannot be found, a null pointer is returned.

7.22.4.7 The quick_exit function

Synopsis

1
```
#include <stdlib.h>
_Noreturn void quick_exit(int status);
```

Description

2 The **quick_exit** function causes normal program termination to occur. No ~~functions~~ function pointers registered by the **atexit** function or signal handlers registered by the **signal** function are called. If a program calls the **quick_exit** function more than once, or calls the **exit** function in addition to the **quick_exit** function, the behavior is undefined. If a signal is raised while the **quick_exit** function is executing, the behavior is undefined.

3 The **quick_exit** function first calls all ~~functions~~ function pointers registered by the **at_quick_exit** function, in the reverse order of their registration,[330)] except that a function pointer is called after any previously registered ~~functions~~ function pointers that had already been called at the time it was registered. If, during the call to any such function or function literal, a call to the **longjmp** function is made that would terminate the call to the registered function pointer, the behavior is undefined.

4 Then control is returned to the host environment by means of the function call **_Exit**(status).

Returns

5 The **quick_exit** function cannot return to its caller.

7.22.4.8 The system function

Synopsis

1
```
#include <stdlib.h>
int system(const char *string);
```

Description

2 If string is a null pointer, the **system** function determines whether the host environment has a *command processor*. If string is not a null pointer, the **system** function passes the string pointed to by string to that command processor to be executed in a manner which the implementation shall document; this might then cause the program calling **system** to behave in a non-conforming manner or to terminate.

Returns

3 If the argument is a null pointer, the **system** function returns nonzero only if a command processor is available. If the argument is not a null pointer, and the **system** function does return, it returns an

[329)] Many implementations provide non-standard functions that modify the environment list.

[330)] Each function pointer is called as many times as it was registered, and in the correct order with respect to other registered ~~functions~~function pointers.

implementation-defined value.

7.22.5 Searching and sorting utilities

1 These utilities make use of a comparison function or function literal to search or sort arrays of unspecified type. Where an argument declared as **size_t** nmemb specifies the length of the array for a function, nmemb can have the value zero on a call to that function; the comparison function or function literal is not called, a search finds no matching element, and sorting performs no rearrangement. Pointer arguments on such a call shall still have valid values, as described in 7.1.4.

2 The implementation shall ensure that the second argument of the comparison function or function literal (when called from **bsearch**), or both arguments (when called from **qsort**), are pointers to elements of the array.[331] The first argument when called from **bsearch** shall equal key.

3 The comparison function or function literal shall not alter the contents of the array. The implementation may reorder elements of the array between calls to the comparison function or function literal, but shall not alter the contents of any individual element.

4 When the same objects (consisting of size bytes, irrespective of their current positions in the array) are passed more than once to the comparison function or function literal, the results shall be consistent with one another. That is, for **qsort** they shall define a total ordering on the array, and for **bsearch** the same object shall always compare the same way with the key.

5 A sequence point occurs immediately before and immediately after each call to the comparison function or function literal, and also between any call to the comparison function or function literal and any movement of the objects passed as arguments to that call.

7.22.5.1 The bsearch function

Synopsis

1
```
#include <stdlib.h>
void *bsearch(const void *key, const void *base,
      size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
```

Description

2 The **bsearch** function searches an array of nmemb objects, the initial element of which is pointed to by base, for an element that matches the object pointed to by key. The size of each element of the array is specified by size.

3 The comparison function or function literal pointed to by compar is called with two arguments that point to the key object and to an array element, in that order. ~~The function~~ A function call shall return an integer less than, equal to, or greater than zero if the key object is considered, respectively, to be less than, to match, or to be greater than the array element. The array shall consist of: all the elements that compare less than, all the elements that compare equal to, and all the elements that compare greater than the key object, in that order.[332]

Returns

4 The **bsearch** function returns a pointer to a matching element of the array, or a null pointer if no match is found. If two elements compare as equal, which element is matched is unspecified.

[331] That is, if the value passed is p, then the following expressions are always nonzero:

```
((char *)p - (char *)base) % size == 0
(char *)p >= (char *)base
(char *)p  <  (char *)base + nmemb * size
```

[332] In practice, the entire array is sorted according to the comparison function.

7.22.5.2 The `qsort` function

Synopsis

1
```
#include <stdlib.h>
void qsort(void *base, size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
```

Description

2 The **`qsort`** function sorts an array of `nmemb` objects, the initial element of which is pointed to by `base`. The size of each object is specified by `size`.

3 The contents of the array are sorted into ascending order according to a comparison function or function literal pointed to by `compar`, which is called with two arguments that point to the objects being compared. ~~The function~~ A function call shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

4 If two elements compare as equal, their order in the resulting sorted array is unspecified.

Returns

5 The **`qsort`** function returns no value.

7.22.6 Integer arithmetic functions

7.22.6.1 The `abs`, `labs` and `dlabs` functions

Synopsis

1
```
#include <stdlib.h>
int abs(int j);
long int labs(long int j);
long long int llabs(long long int j);
```

Description

2 The **`abs`**, **`labs`**, and **`llabs`** functions compute the absolute value of an integer `j`. If the result cannot be represented, the behavior is undefined.[333)]

Returns

3 The **`abs`**, **`labs`**, and **`llabs`**, functions return the absolute value.

7.22.6.2 The `div`, `ldiv`, and `lldiv` functions

Synopsis

1
```
#include <stdlib.h>
div_t div(int numer, int denom);
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
```

Description

2 The **`div`**, **`ldiv`**, and **`lldiv`**, functions compute `numer/denom` and `numer%denom` in a single operation.

Returns

3 The **`div`**, **`ldiv`**, and **`lldiv`** functions return a structure of type **`div_t`**, **`ldiv_t`**, and **`lldiv_t`**, respectively, comprising both the quotient and the remainder. The structures shall contain (in either order) the members `quot` (the quotient) and `rem` (the remainder), each of which has the same type as the arguments `numer` and `denom`. If either part of the result cannot be represented, the behavior is undefined.

[333)] The absolute value of the most negative number cannot be represented in two's complement.

(6.5.1.1) *generic-selection:*
_Generic **(** *assignment-expression* **,** *generic-assoc-list* **)**

(6.5.1.1) *generic-assoc-list:*
generic-association
generic-assoc-list **,** *generic-association*

(6.5.1.1) *generic-association:*
type-name **:** *assignment-expression*
default **:** *assignment-expression*

(6.5.2) *postfix-expression:*
primary-expression
postfix-expression **[** *expression* **]**
postfix-expression **(** *argument-expression-list*$_{\text{opt}}$ **)**
postfix-expression **.** *identifier*
postfix-expression **->** *identifier*
postfix-expression **++**
postfix-expression **-**
(*type-name* **)** **{** *initializer-list* **}**
(*type-name* **)** **{** *initializer-list* **,** **}**
lambda-expression

(6.5.2) *argument-expression-list:*
assignment-expression
argument-expression-list **,** *assignment-expression*

(6.5.2.6) *lambda-expression:*
capture-clause *parameter-clause*$_{\text{opt}}$ *attribute-specifier-sequence*$_{\text{opt}}$ *function-body*

(6.5.2.6) *capture-clause:*
[*capture-list*$_{\text{opt}}$ **]**

(6.5.2.6) *capture-list:*
capture-list-element
capture-list **,** *capture-list-element*

(6.5.2.6) *capture-list-element:*
value-capture
identifier-capture

(6.5.2.6) *value-capture:*
capture **=** *assignment-expression*

(6.5.2.6) *identifier-capture:*
& *capture*

(6.5.2.6) *capture:*
identifier

(6.5.2.6) *parameter-clause:*
(*parameter-list*$_{\text{opt}}$ **)**

(6.5.3) *unary-expression:*
postfix-expression
++ *unary-expression*
- *unary-expression*
unary-operator *cast-expression*
sizeof *unary-expression*
sizeof **(** *type-name* **)**
_Alignof **(** *type-name* **)**